

Windows Firewall Configuration & System Hardening

PROJECT REPORT

Omkar Shinde

B. Tech Computer Science Engineering
(Cyber Security)

Cognetix Technology

Intern ID – CT/CS/X146

Cyber Security Internship Program

Assessment Date: June 2026

Confidentiality Statement

This report has been prepared as part of the Cognetix Technology Cyber Security Internship Program. All assessment activities were conducted on systems owned by or explicitly authorized for testing by the author. The assessment was performed within a controlled laboratory environment using safe and non-intrusive techniques. No unauthorized systems or third-party networks were scanned during this project.

DECLARATION

I hereby declare that the project entitled "**Windows Firewall Configuration & System Hardening**" is an original work carried out by me as part of the **Cyber Security Internship Program at Cognetix Technology**.

This report has been prepared based on practical implementation, testing, verification, and documentation performed during the internship period. The work presented in this report has not been submitted previously, either in full or in part, for the award of any degree, diploma, certification, or internship completion at any other institution or organization.

All references, documentation, tools, and resources used during the completion of this project have been appropriately acknowledged wherever applicable. Every firewall rule, security configuration, verification procedure, and system hardening measure described in this report was implemented and tested in a controlled environment on a Windows 11 system.

I further declare that the information contained in this report is true and accurate to the best of my knowledge.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Cognetix Technology** for providing me with the opportunity to undertake this Cyber Security Internship and work on the project "**Windows Firewall Configuration & System Hardening**." This internship has provided me with valuable practical exposure to enterprise security concepts, Windows security management, firewall configuration, and endpoint hardening techniques.

I extend my heartfelt thanks to the mentors and the Cognetix Technology team for designing a structured internship program that encouraged practical implementation, documentation, and continuous learning. The project enabled me to understand the importance of securing operating systems by implementing firewall rules, verifying system security configurations, and applying industry-recommended hardening practices.

I would also like to acknowledge the valuable technical resources and documentation provided by **Microsoft**, **NIST (National Institute of Standards and Technology)**, **CIS (Centre for Internet Security)**, and the **OWASP Foundation**. Their publicly available security guidelines, recommendations, and best practices greatly contributed to the successful completion of this project.

Finally, I express my sincere gratitude to my faculty members, my university, and everyone who supported and motivated me throughout this internship. This project has significantly enhanced my practical cybersecurity knowledge and strengthened my understanding of system security, firewall management, and Windows endpoint protection.

I am thankful to everyone who contributed, directly or indirectly, to the successful completion of this project.

"Learning never exhausts the mind; it only strengthens our ability to secure the digital world."

Omkar Shinde

Cyber Security Intern

Cognetix Technology

ABSTRACT

In today's digital landscape, endpoint security plays a critical role in protecting systems against unauthorized access, malware, and cyber threats. Proper firewall configuration and operating system hardening are essential security practices that help reduce the attack surface and strengthen the overall security posture of an organization's devices. This project, **"Windows Firewall Configuration & System Hardening,"** was undertaken as part of the **Cyber Security Internship Program at Cognetix Technology** to demonstrate the implementation of practical security controls on a Windows 11 operating system.

The primary objective of this project was to configure the Windows Defender Firewall by creating custom inbound firewall rules, allowing only essential network services while blocking insecure and unnecessary ports. Five custom firewall rules were implemented to allow secure services such as **HTTP (Port 80)**, **HTTPS (Port 443)**, and **SSH (Port 22)** while blocking insecure services including **FTP (Port 21)** and **Telnet (Port 23)**. These firewall rules were verified using both Windows PowerShell and Command Prompt utilities to ensure accurate implementation and effectiveness.

In addition to firewall configuration, several Windows security hardening techniques were applied to improve endpoint security. The system was evaluated to verify the status of Microsoft Defender Antivirus, Windows Defender Firewall, Real-Time Protection, Secure Boot, Trusted Platform Module (TPM), Core Isolation, Memory Integrity, Device Encryption, User Account Control (UAC), Windows Update, and the built-in Guest account. These security features collectively contribute to strengthening system resilience against common cyber threats and unauthorized access.

The project also included detailed documentation, security verification, testing procedures, and evidence collection through screenshots and command-line outputs. Microsoft PowerShell commands, Windows Security settings, and firewall management tools were used to validate the successful implementation of all security controls. The project demonstrates the practical application of Windows security best practices and enterprise endpoint hardening techniques commonly adopted by organizations to protect corporate workstations.

Overall, this project enhanced practical knowledge of Windows security administration, firewall management, endpoint protection, and system hardening while reinforcing the importance of implementing layered security controls to safeguard modern computing environments.

Keywords: Windows Defender Firewall, Windows Security, System Hardening, Endpoint Security, Firewall Rules, Microsoft Defender Antivirus, Secure Boot, TPM, Device Encryption, User Account Control, Cyber Security, Windows 11.

TABLE OF CONTENTS

DECLARATION	2
ACKNOWLEDGEMENT	3
ABSTRACT	4
TABLE OF CONTENTS.....	5
 CHAPTER 1: INTRODUCTION	 7
1.1 Project Overview	8
1.2 Problem Statement	9
1.3 Objectives	9
1.4 Scope of the Project	10
1.5 Expected Outcomes	11
 CHAPTER 2: LITERATURE REVIEW & TECHNICAL BACKGROUND	 12
2.1 Windows Defender Firewall	12
2.2 Windows Security	13
2.3 System Hardening	14
2.4 Windows Security Features	15
2.5 CIA Triad	16
2.6 Defense-in-Depth Model	16
 CHAPTER 3: SYSTEM ENVIRONMENT & TOOLS USED	 17
3.1 Hardware Specifications	17
3.2 Software Specifications	18
3.3 Network Configuration	20
3.4 Security Tools Used	22
 CHAPTER 4: FIREWALL CONFIGURATION & SYSTEM HARDENING IMPLEMENTATION	 24
4.1 Implementation Methodology	24
4.2 Baseline Firewall Assessment	25
4.3 Custom Firewall Rule Configuration	27
4.4 Blocking Insecure Network Services	32
4.5 Complete Firewall Rule Summary	35
4.6 Firewall Verification and Validation	36
4.7 Windows System Hardening Implementation	41
 CHAPTER 5: RESULTS, ANALYSIS AND SECURITY ASSESSMENT	 51
5.1 Firewall Configuration Results	51
5.2 Windows Security Hardening Results	52
5.3 Security Improvements Achieved	52
5.4 Security Score Assessment	53
5.5 Discussion	54

CHAPTER 6: SECURITY BEST PRACTICES & RECOMMENDATIONS	55
6.1 Firewall Management Best Practices	55
6.2 Windows System Hardening Best Practices	56
6.3 Enterprise Security Recommendations	56
6.4 Mapping to Industry Standards	57
6.5 Future Enhancements	58
CHAPTER 7: CHALLENGES, LEARNING OUTCOMES AND CONCLUSION	59
7.1 Challenges Faced	59
7.2 Learning Outcomes	60
7.3 Project Outcomes	61
7.4 Conclusion	61
REFERENCES	63
APPENDIX	64
Appendix A – Commands Used	64
Appendix B – Abbreviations	65

CHAPTER 1

1. INTRODUCTION

Cybersecurity has become one of the most critical aspects of modern computing as organizations increasingly depend on digital infrastructure to conduct business operations. With the rapid growth of internet-connected devices, cloud computing, and remote work environments, operating systems have become prime targets for cyberattacks. Threats such as malware, ransomware, unauthorized access, brute-force attacks, and network exploitation continue to evolve, making it essential for organizations to implement effective security controls that protect their systems from compromise.

One of the primary security mechanisms used to safeguard endpoint devices is the host-based firewall. A firewall monitors and controls incoming and outgoing network traffic based on predefined security rules. Proper firewall configuration prevents unauthorized access, restricts unnecessary network services, and significantly reduces the attack surface of a system. However, firewall configuration alone is not sufficient to secure an endpoint. Organizations also implement operating system hardening techniques to disable unnecessary services, enable built-in security features, enforce authentication policies, and strengthen system defences against potential attacks.

Windows 11 provides numerous built-in security features including Microsoft Defender Firewall, Microsoft Defender Antivirus, Secure Boot, Trusted Platform Module (TPM), Device Encryption, User Account Control (UAC), Memory Integrity, Core Isolation, and Real-Time Protection. When configured correctly, these features provide multiple layers of defence that help protect enterprise workstations from both internal and external threats.

This project focuses on implementing Windows Defender Firewall configuration and Windows operating system hardening using native Windows security features. The project demonstrates how firewall rules can be configured to permit only essential services while blocking insecure network ports. In addition, several security hardening techniques are applied and verified using graphical Windows security settings and PowerShell-based validation commands.

The project follows practical cybersecurity methodologies commonly adopted by enterprise IT security teams to improve endpoint protection and strengthen the security posture of Windows systems.

1.1 Project Overview

The **Windows Firewall Configuration & System Hardening** project was carried out as part of the Cyber Security Internship Program at **Cognetix Technology**. The project focuses on strengthening the security of a Windows 11 workstation by implementing firewall rules, verifying built-in Windows security controls, and applying system hardening techniques.

The project begins with collecting baseline system information to understand the existing security configuration of the operating system. Windows Defender Firewall is then configured by creating custom inbound firewall rules that allow secure services while blocking insecure and unnecessary ports. Following firewall configuration, multiple Windows security components are evaluated, including Microsoft Defender Antivirus, Secure Boot, TPM, Core Isolation, Memory Integrity, Device Encryption, User Account Control, Windows Update, and Guest Account security.

The implemented configurations are validated using Windows PowerShell, Command Prompt, and Windows Security, ensuring that each security control functions correctly and contributes to the overall protection of the system.

Parameter	Details
Project Title	Windows Firewall Configuration & System Hardening
Internship	Cyber Security Internship
Organization	Cognetix Technology
Operating System	Windows 11 Home Single Language
Firewall	Microsoft Defender Firewall
Primary Objective	Configure Firewall and Harden Windows Security
Tools Used	Windows Security, PowerShell, Command Prompt

Table 1.1 Project Overview

1.2 Problem Statement

As cyber threats continue to evolve, Windows-based systems have become primary targets for attackers attempting to gain unauthorized access, steal sensitive information, or disrupt business operations. Many security incidents occur because systems are deployed with default configurations, unnecessary services remain enabled, insecure network ports are left exposed, or built-in security features are not properly configured.

Organizations require a secure baseline configuration that minimizes security risks while maintaining system functionality. Proper firewall configuration and operating system hardening significantly reduce the attack surface by restricting unauthorized network communication, enforcing secure authentication mechanisms, enabling built-in security controls, and protecting critical system resources.

This project addresses these security challenges by implementing Windows Defender Firewall configuration together with enterprise-level Windows security hardening techniques. The project demonstrates how multiple layers of security can work together to improve endpoint protection and reduce the likelihood of successful cyberattacks.

1.3 Objectives

The primary objective of this project is to strengthen the security posture of a Windows 11 workstation by implementing firewall configuration and system hardening techniques using native Windows security features.

The specific objectives are as follows:

- To collect baseline system information before implementing security controls.
- To configure Windows Defender Firewall by creating custom inbound firewall rules.
- To allow essential services including HTTP, HTTPS, and SSH while blocking insecure services such as FTP and Telnet.
- To verify firewall rules using Windows PowerShell and Command Prompt.
- To evaluate Microsoft Defender Antivirus and verify Real-Time Protection.
- To verify Secure Boot, Trusted Platform Module (TPM), and Device Encryption.
- To enable and verify Memory Integrity and Local Security Authority (LSA) Protection.
- To validate User Account Control (UAC) configuration.
- To verify Windows Update functionality.
- To confirm that the built-in Guest account is disabled.
- To document all security configurations with screenshots and technical evidence.

Objective	Description
Firewall Configuration	Configure Windows Defender Firewall
Port Management	Allow secure ports and block insecure ports
Windows Hardening	Enable and verify built-in Windows security features
Verification	Validate all security configurations using PowerShell and CMD
Documentation	Capture screenshots and technical evidence
Security Improvement	Strengthen overall endpoint security posture

Table 1.2: Project Objectives

1.4 Scope of the Project

The scope of this project is limited to configuring and hardening a Windows 11 workstation using native Microsoft security technologies. The implementation focuses on Windows Defender Firewall configuration, Microsoft Defender Antivirus verification, Secure Boot validation, Trusted Platform Module (TPM) verification, Device Encryption, User Account Control (UAC), Windows Update, Memory Integrity, and Guest account security.

The project does not include penetration testing, vulnerability exploitation, malware analysis, or offensive security activities. Instead, the focus is on preventive security measures that improve endpoint protection through secure configuration and system hardening.

All firewall rules and security configurations were implemented and tested within a controlled local environment without affecting production systems or external network infrastructure.

1.5 Expected Outcomes

Upon completion of this project, the Windows 11 system is expected to achieve an improved security posture through the implementation of multiple defensive security controls.

The expected outcomes include:

- Improved protection against unauthorized network access.
- Reduced exposure to insecure network services.
- Enhanced endpoint security through Windows Defender Firewall.
- Verification of Microsoft Defender Antivirus functionality.
- Secure boot process validation.
- Protection of user credentials through LSA Protection.
- Increased resistance against kernel-level attacks using Memory Integrity.
- Encryption of sensitive data using Device Encryption.
- Improved compliance with Windows security best practices.
- Comprehensive documentation of all implemented security controls.

Security Area	Expected Result
Firewall	Secure inbound communication
Endpoint Protection	Microsoft Defender enabled
System Integrity	Secure Boot and TPM verified
Authentication	UAC and Guest Account secured
Data Protection	Device Encryption enabled
Overall Security	Reduced attack surface

Table 1.3 Expected Security Outcomes

CHAPTER 2

2. LITERATURE REVIEW & TECHNICAL BACKGROUND

Modern organizations rely heavily on endpoint security mechanisms to protect systems against unauthorized access, malware infections, privilege escalation attacks, and network-based threats. Windows operating systems provide multiple built-in security features that work together to create a layered defense strategy. Among these security mechanisms, Windows Defender Firewall and Windows Security are fundamental components responsible for controlling network communication and protecting the operating system from external attacks.

System hardening is the process of reducing the attack surface of a computer by configuring security settings, disabling unnecessary services, enforcing strong authentication mechanisms, and enabling protective technologies that improve the overall resilience of the operating system. Enterprise organizations commonly implement security hardening frameworks based on industry standards such as the Centre for Internet Security (CIS) Benchmarks, Microsoft Security Baselines, and the National Institute of Standards and Technology (NIST) Cybersecurity Framework.

This chapter discusses the fundamental concepts related to Windows Defender Firewall, system hardening, layered security architecture, and Windows security features that form the technical foundation of this project.

2.1 Windows Defender Firewall

Windows Defender Firewall is a host-based firewall integrated into Microsoft Windows operating systems. It monitors inbound and outbound network traffic and determines whether specific network connections should be allowed or blocked according to predefined security rules. Unlike traditional perimeter firewalls that protect an entire network, Windows Defender Firewall protects individual endpoints by filtering traffic directly on the host system.

The firewall supports multiple security profiles, including Domain, Private, and public profiles, allowing administrators to apply different security policies depending on the network environment. Administrators can create inbound and outbound rules based on applications, ports, protocols, IP addresses, or predefined Windows services.

Proper firewall configuration significantly reduces the attack surface of a system by limiting unnecessary network communication while allowing only trusted services to operate.

Profile	Usage	Security Level	Typical Environment
Domain	Corporate Network	High	Active Directory Domain
Private	Home / Trusted Network	Medium	Home or Office
Public	Public Wi-Fi	Very High	Airports, Cafes, Hotels

Table 2.1: Windows Defender Firewall Profiles.

2.2 Windows Security

Windows Security is Microsoft's integrated security platform that provides centralized management of endpoint protection features. It combines Microsoft Defender Antivirus, Windows Defender Firewall, Device Security, App & Browser Control, Family Options, Account Protection, and Device Performance monitoring into a unified interface.

These security components collectively provide real-time malware detection, network protection, exploit mitigation, hardware-based security, credential protection, and application reputation services. Windows Security continuously monitors the operating system and alerts users when security settings require attention.

In enterprise environments, Windows Security forms the first line of defence against malware, ransomware, phishing attacks, unauthorized access, and other endpoint threats.

Component	Purpose	Status in Project
Microsoft Defender Antivirus	Malware Protection	Verified
Firewall & Network Protection	Network Filtering	Configured
Device Security	Hardware Protection	Verified
App & Browser Control	SmartScreen Protection	Verified
Windows Update	Patch Management	Verified
Device Encryption	Data Protection	Enabled

Table 2.2: Windows Security Components.

2.3 System Hardening

System hardening is the process of securing a computer system by reducing its attack surface through the implementation of security best practices. The primary objective of system hardening is to eliminate unnecessary services, restrict unauthorized access, strengthen authentication mechanisms, enable built-in security features, and minimize potential vulnerabilities that could be exploited by attackers.

Windows operating systems provide several native security mechanisms that contribute to system hardening, including Microsoft Defender Firewall, Microsoft Defender Antivirus, User Account Control (UAC), Device Encryption, Secure Boot, Trusted Platform Module (TPM), Core Isolation, and Memory Integrity. These technologies work together to protect the operating system against malware infections, privilege escalation, unauthorized access, and credential theft.

A properly hardened Windows workstation significantly improves endpoint security and reduces the likelihood of successful cyberattacks while maintaining system usability and performance

Security Control	Purpose	Status in Project
Windows Defender Firewall	Network Protection	Configured
Microsoft Defender Antivirus	Malware Protection	Verified
User Account Control (UAC)	Privilege Management	Verified
Secure Boot	Boot Integrity	Verified
Trusted Platform Module (TPM)	Hardware Security	Verified
Device Encryption	Data Protection	Enabled
Memory Integrity	Kernel Protection	Enabled
Windows Update	Patch Management	Verified

Table 2.3: Windows Security Hardening Techniques

2.4 Windows Security Features

Windows 11 incorporates multiple security technologies designed to provide layered protection against modern cyber threats. Unlike traditional security models that rely on a single defence mechanism, Windows implements a defence-in-depth approach where multiple independent security controls operate together to protect the operating system.

During this project, several Windows security features were verified and documented, including Microsoft Defender Antivirus, Windows Defender Firewall, Device Encryption, Secure Boot, Core Isolation, Memory Integrity, Local Security Authority (LSA) Protection, User Account Control (UAC), and Windows Update. Each feature contributes to protecting different aspects of the operating system, thereby improving the overall security posture.

Table 2.4: Windows Security Features Verified During the Project

Feature	Purpose	Verification Method	Status
Firewall	Network Protection	Windows Security	Verified
Antivirus	Malware Detection	Microsoft Defender	Verified
Secure Boot	Boot Protection	Device Security	Enabled
TPM	Hardware Security	Device Security	Present
Core Isolation	Kernel Protection	Windows Security	Enabled
Memory Integrity	Driver Protection	Core Isolation	Enabled
Device Encryption	Data Security	Settings	Enabled
UAC	Privilege Control	UAC Settings	Enabled
Windows Update	Patch Management	Settings	Verified

Table 2.4: Windows Security Features Verified During the Project

2.5 CIA Triad

The **Confidentiality, Integrity, and Availability (CIA) Triad** is a fundamental cybersecurity model used to evaluate and implement security controls within information systems.

- **Confidentiality** ensures that sensitive information is accessible only to authorized users. Features such as Device Encryption and User Account Control contribute to maintaining confidentiality.
- **Integrity** ensures that system files and data remain accurate, authentic, and protected from unauthorized modification. Secure Boot, TPM, and Memory Integrity help preserve system integrity.
- **Availability** ensures that systems and services remain accessible to authorized users whenever required. Windows Defender Firewall, Windows Update, and Microsoft Defender Antivirus contribute to maintaining system availability by protecting against disruptions caused by malware or unauthorized access.

2.6 Défense-in-Depth Model

Défense-in-Depth is a cybersecurity strategy that uses multiple layers of security controls to protect systems against attacks. Instead of relying on a single security mechanism, organizations deploy several independent protective technologies so that if one control fails, others continue to provide protection.

In this project, Défense-in-Depth was implemented using Microsoft Defender Firewall, Microsoft Defender Antivirus, Secure Boot, TPM, Core Isolation, Device Encryption, User Account Control, Windows Update, and Guest Account restrictions. These security controls collectively provide network security, malware protection, hardware security, authentication, data protection, and operating system integrity.

This chapter presented the theoretical concepts supporting the project, including Windows Defender Firewall, Windows security features, system hardening techniques, the CIA Triad, and the Défense-in-Depth security model. These concepts provide the foundation for the practical implementation described in the next chapter.

CHAPTER 3

3. SYSTEM ENVIRONMENT & TOOLS USED

The successful implementation of firewall configuration and system hardening requires a secure and properly configured environment. Before making any security-related changes, the target system was analysed to collect baseline information including hardware specifications, operating system details, network configuration, and existing security settings. This baseline assessment ensured that all security modifications could be validated and documented accurately.

The implementation was performed on a Windows 11 laptop using built-in Microsoft security features and administrative tools. No third-party firewall or endpoint protection software was used during the project. All firewall rules, system hardening measures, and verification activities were carried out using native Windows utilities.

3.1 Hardware Specifications

The project was implemented on an HP laptop running Windows 11 Home Single Language. The device is equipped with an AMD Ryzen 7 processor, providing sufficient processing capability to support Windows security services, Microsoft Defender, virtualization features, and firewall management.

The hardware platform also supports modern security technologies such as Secure Boot, Trusted Platform Module (TPM), Device Encryption, and Memory Integrity, making it suitable for implementing enterprise-level endpoint security controls.

Component	Specification
Laptop Model	HP ENVY x360 Convertible 13-ay0xxx
Processor	AMD Ryzen 7 4700U
Operating System	Windows 11 Home Single Language
Architecture	64-bit Operating System
Security Features	Secure Boot, TPM, Device Encryption
User Account	HP (Administrator)

Table 3.1: Hardware Specifications

Windows Firewall Configuration & System Hardening

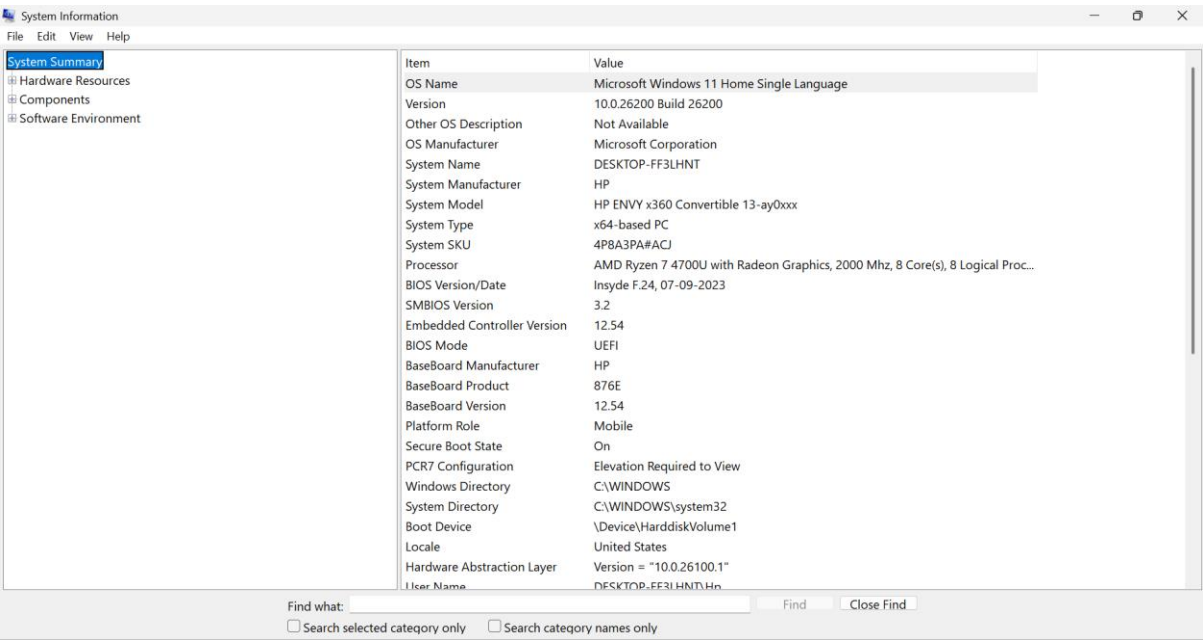


Figure 3.1: Windows System Information (Baseline Assessment)

Figure 3.1 displays the baseline system information collected before implementing firewall configuration and security hardening. The information includes the operating system version, hardware platform, processor details, firmware configuration, and security-related system properties. Establishing this baseline enables comparison of the system state before and after implementing security controls.

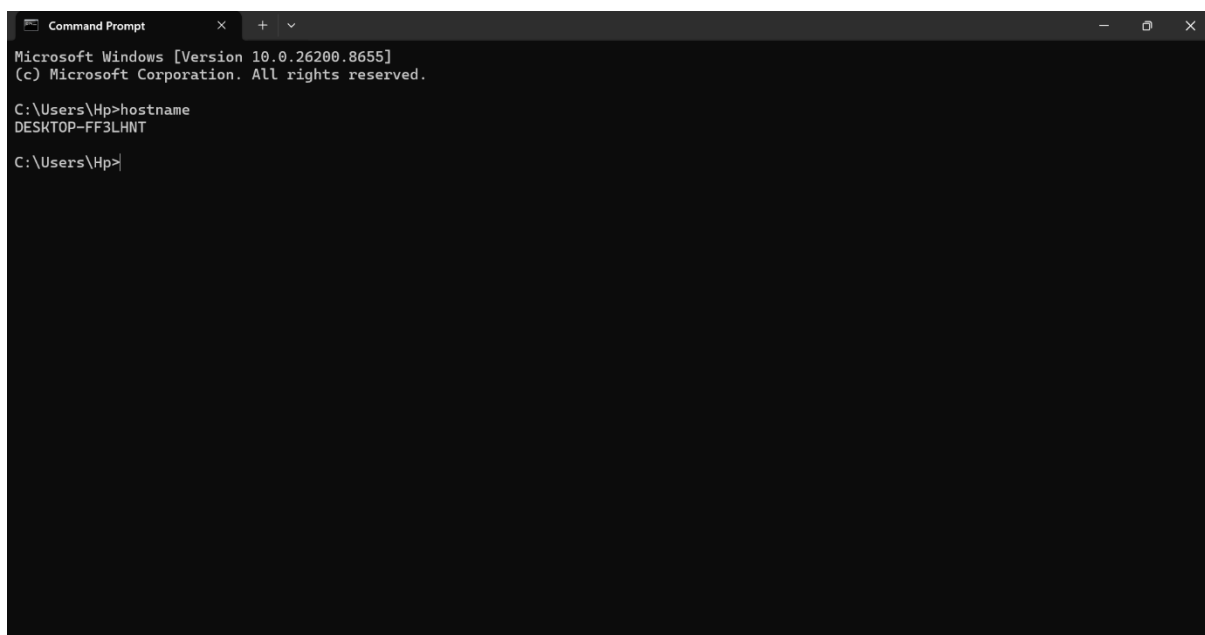
3.2 Software Specifications

The implementation relied entirely on native Microsoft Windows security components. Using built-in security tools ensures compatibility with the operating system while demonstrating enterprise security practices without introducing additional software dependencies.

The primary software components used during this project include Windows Security, Windows Defender Firewall, Windows PowerShell, Command Prompt, and Windows Update. These tools provide comprehensive capabilities for configuring, managing, and verifying Windows security settings.

Software / Tool	Version / Platform	Purpose
Windows 11	Home Single Language	Operating System
Windows Defender Firewall	Built-in	Firewall Management
Windows Security	Built-in	Endpoint Protection
Windows PowerShell	Built-in	Verification & Administration
Command Prompt	Built-in	Network & Firewall Commands
Windows Update	Built-in	Patch Management

Table 3.2: Software Environment



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

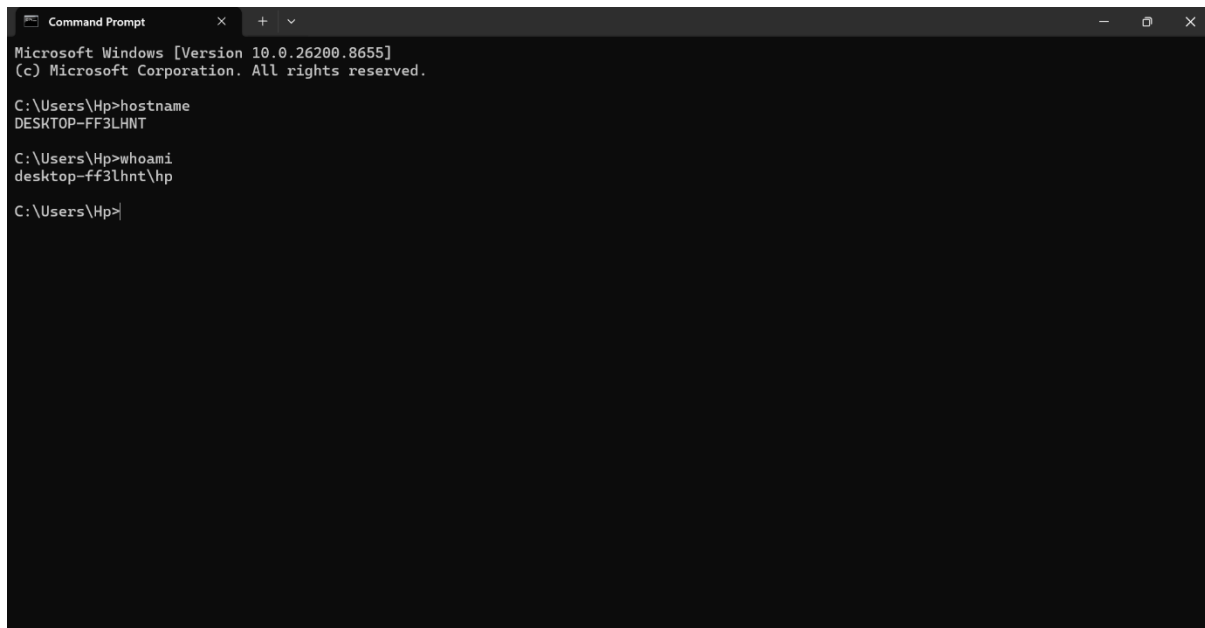
C:\Users\Hp>hostname
DESKTOP-FF3LHNT

C:\Users\Hp>
```

Figure 3.2: Hostname Verification Using Command Prompt

Explanation:

The hostname command was executed to identify the unique system name assigned to the workstation. This information is useful for documenting the assessment environment and identifying the target device during system administration and security auditing.

A screenshot of a Windows Command Prompt window. The title bar reads 'Command Prompt'. The window content shows the following text: 'Microsoft Windows [Version 10.0.26200.8655] (c) Microsoft Corporation. All rights reserved. C:\Users\Hp>hostname DESKTOP-FF3LHNT C:\Users\Hp>whoami desktop-ff3lhnt\hp C:\Users\Hp>'. The background is black, and the text is white.

```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Hp>hostname
DESKTOP-FF3LHNT

C:\Users\Hp>whoami
desktop-ff3lhnt\hp

C:\Users\Hp>
```

Figure 3.3: Current Logged-in User Verification

Explanation:

The whoami command was used to verify the currently logged-in user account. Administrative privileges are required to configure firewall rules, modify security settings, and execute PowerShell commands used throughout this project.

3.3 Network Configuration

Before implementing firewall rules and security hardening measures, the network configuration of the target system was documented to establish a baseline. Network information was collected using the Windows Command Prompt utility (ipconfig /all), which provides detailed information about network adapters, IP addressing, DNS configuration, DHCP status, and gateway information.

Documenting the network configuration is important because firewall rules are applied to network interfaces and influence how inbound and outbound traffic is processed. This information also assists in troubleshooting firewall behaviour and verifying network connectivity after security configurations are applied.

Windows Firewall Configuration & System Hardening

Parameter	Value
Host Name	DESKTOP-FF3LHNT
Network Adapter	Intel/Realtek Wireless Adapter
IPv4 Address	192.168.1.35
Default Gateway	192.168.1.1
DHCP	Enabled
DNS Server	As assigned by DHCP
VMware Adapters	VMnet1 and VMnet8
Network Type	Private Network

Table 3.3: Network Configuration

```
C:\Users\Hmpipconfig /all
Windows IP Configuration

Host Name . . . . . : DESKTOP-FF3LHNT
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled . . . . . : No
WINS Proxy Enabled . . . . . : No

Ethernet adapter vEthernet (WSL (Hyper-V firewall)):

Connection-specific DNS Suffix . . :
Description . . . . . : Hyper-V Virtual Ethernet Adapter
Physical Address. . . . . : 00-15-5D-C1-01-09
DHCP Enabled. . . . . : No
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::9913:1c06:4e50:b528%10(Preferred)
IPv4 Address. . . . . : 172.26.224.11(Preferred)
Subnet Mask . . . . . : 255.255.240.0
Default Gateway . . . . . :
DHCPv6 IAID . . . . . : 871894189
DHCPv6 Client DUID. . . . . : 00-01-00-00-2D-9C-19-98-4B-E7-DA-17-F7-DD
NetBIOS over Tcpip. . . . . : Enabled

Wireless LAN adapter Local Area Connection* 1:

Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . . :
Description . . . . . : Microsoft Wi-Fi Direct Virtual Adapter
Physical Address. . . . . : 4A-E7-DA-17-F7-DD
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes

Wireless LAN adapter Local Area Connection* 2:

Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . . :
Description . . . . . : Microsoft Wi-Fi Direct Virtual Adapter #2
Physical Address. . . . . : CA-E7-DA-17-F7-DD
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes

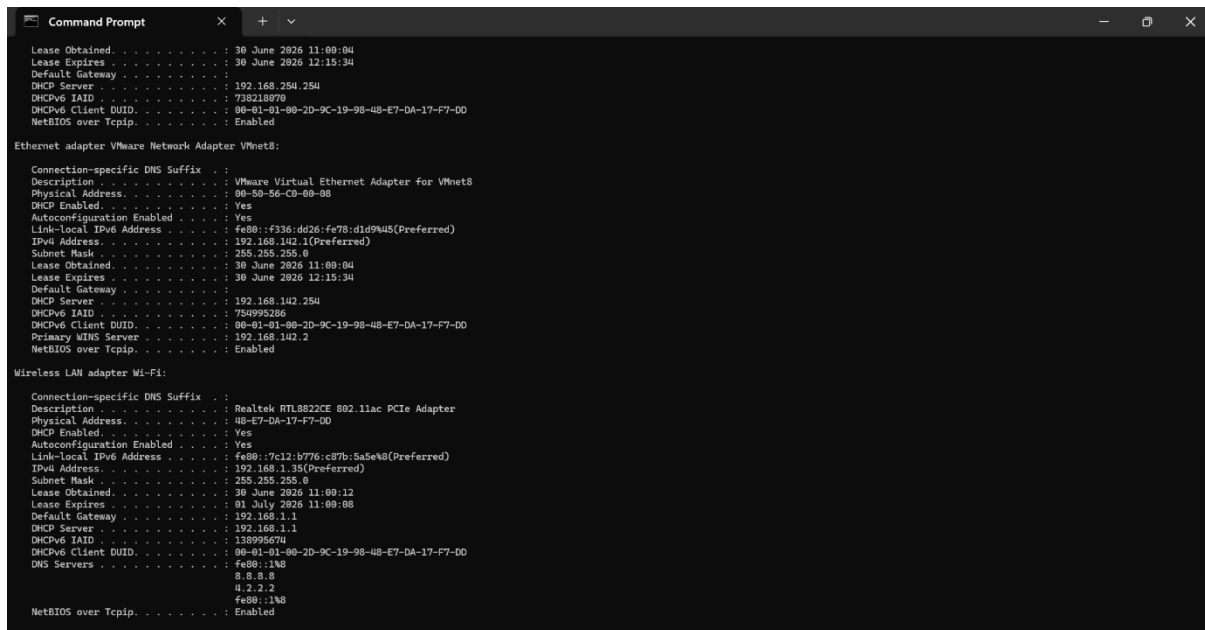
Ethernet adapter VMware Network Adapter VMnet1:

Connection-specific DNS Suffix . . :
Description . . . . . : VMware Virtual Ethernet Adapter for VMnet1
Physical Address. . . . . : 00-50-56-C0-00-01
```

Figure 3.4: Network Configuration Using ipconfig /all (Part 1)

Explanation:

Figure 3.4 displays the first section of the network configuration collected using the ipconfig /all command. The output includes the host name, DNS suffix, node type, IP routing status, and details of the installed network adapters. This information establishes the baseline network environment prior to firewall configuration.



```
Command Prompt
Lease Obtained. . . . . : 30 June 2026 11:00:04
Lease Expires . . . . . : 30 June 2026 12:15:34
Default Gateway . . . . . :
DHCP Server . . . . . : 192.168.254.254
DHCPv6 IAID . . . . . : 738218870
DHCPv6 Client DUID. . . . . : 00-01-00-2D-9C-19-98-48-E7-DA-17-F7-00
NetBIOS over Tcpip. . . . . : Enabled

Ethernet adapter VMware Network Adapter VMnet8:

Connection-specific DNS Suffix . :
Description . . . . . : VMware Virtual Ethernet Adapter for VMnet8
Physical Address. . . . . : 00-50-56-CB-80-80
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::f336:dd26:f78:d1d9%45(Preferred)
IPv4 Address. . . . . : 192.168.142.1(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : 30 June 2026 11:00:04
Lease Expires . . . . . : 30 June 2026 12:15:34
Default Gateway . . . . . :
DHCP Server . . . . . : 192.168.142.254
DHCPv6 IAID . . . . . : 754995286
DHCPv6 Client DUID. . . . . : 00-01-00-2D-9C-19-98-48-E7-DA-17-F7-00
Primary WINS Server . . . . . : 192.168.142.2
NetBIOS over Tcpip. . . . . : Enabled

Wireless LAN adapter Wi-Fi:

Connection-specific DNS Suffix . :
Description . . . . . : Realtek RTL8822CE 802.11ac PCIe Adapter
Physical Address. . . . . : 40-E7-DA-17-F7-00
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::7c12:b776:c97b:5a5e%48(Preferred)
IPv4 Address. . . . . : 192.168.1.35(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : 30 June 2026 11:00:12
Lease Expires . . . . . : 01 July 2026 11:00:08
Default Gateway . . . . . : 192.168.1.1
DHCP Server . . . . . : 192.168.1.1
DHCPv6 IAID . . . . . : 138995674
DHCPv6 Client DUID. . . . . : 00-01-00-2D-9C-19-98-48-E7-DA-17-F7-00
DNS Servers . . . . . : fe80::1%8
                        8.8.8.8
                        4.2.2.2
                        fe80::1%8
NetBIOS over Tcpip. . . . . : Enabled
```

Figure 3.5: Network Configuration Using ipconfig /all (Part 2)

Explanation:

The second section of the network configuration output provides details about the active wireless network interface, including IPv4 address assignment, subnet mask, default gateway, DHCP lease information, and DNS server configuration. These settings confirm successful network connectivity and support subsequent firewall testing.

3.4 Security Tools Used

The project utilized only built-in Microsoft Windows administrative and security tools. These native utilities provide comprehensive capabilities for configuring firewall rules, verifying security controls, managing system settings, and validating the effectiveness of implemented security measures.

The use of built-in tools demonstrates that enterprise-level endpoint hardening can be performed without relying on third-party software while maintaining compatibility with Windows security architecture.

Tool	Category	Purpose	Usage
Windows Security	GUI	Manage Windows security features	Security verification
Windows Defender Firewall	GUI	Configure firewall rules	Firewall management
Windows PowerShell	CLI	Execute verification commands	Rule validation
Command Prompt	CLI	Network and firewall commands	System verification
Windows Update	GUI	Install security updates	Patch management
Device Security	GUI	Verify Secure Boot and TPM	Hardware security
Settings App	GUI	Configure Device Encryption and UAC	System hardening

Table 3.4: Security Tools Used During the Project

Chapter Summary

This chapter described the system environment, hardware and software specifications, network configuration, and security tools used during the implementation of the project. Establishing a baseline environment before configuring the firewall and applying system hardening measures ensures accurate documentation, repeatability, and verification of all security controls.

CHAPTER 4

4. FIREWALL CONFIGURATION & SYSTEM HARDENING IMPLEMENTATION

This chapter describes the practical implementation of Windows Defender Firewall configuration and Windows 11 system hardening performed during this project. The implementation followed a structured methodology beginning with a baseline assessment of the operating system, followed by firewall configuration, creation of custom firewall rules, verification of security settings, and implementation of operating system hardening measures.

All configurations were performed using native Microsoft Windows administrative tools, including Windows Defender Firewall with Advanced Security, Windows Security, Windows PowerShell, Command Prompt, and Windows Settings. After each implementation step, verification procedures were performed to ensure that the security controls were functioning correctly.

The objective of this implementation was to improve endpoint security by restricting unauthorized network access, protecting critical operating system components, and enabling multiple layers of built-in Windows security features.

4.1 Implementation Methodology

The project followed a systematic implementation process to ensure that all security configurations were properly documented and verified.

The implementation methodology consisted of the following stages:

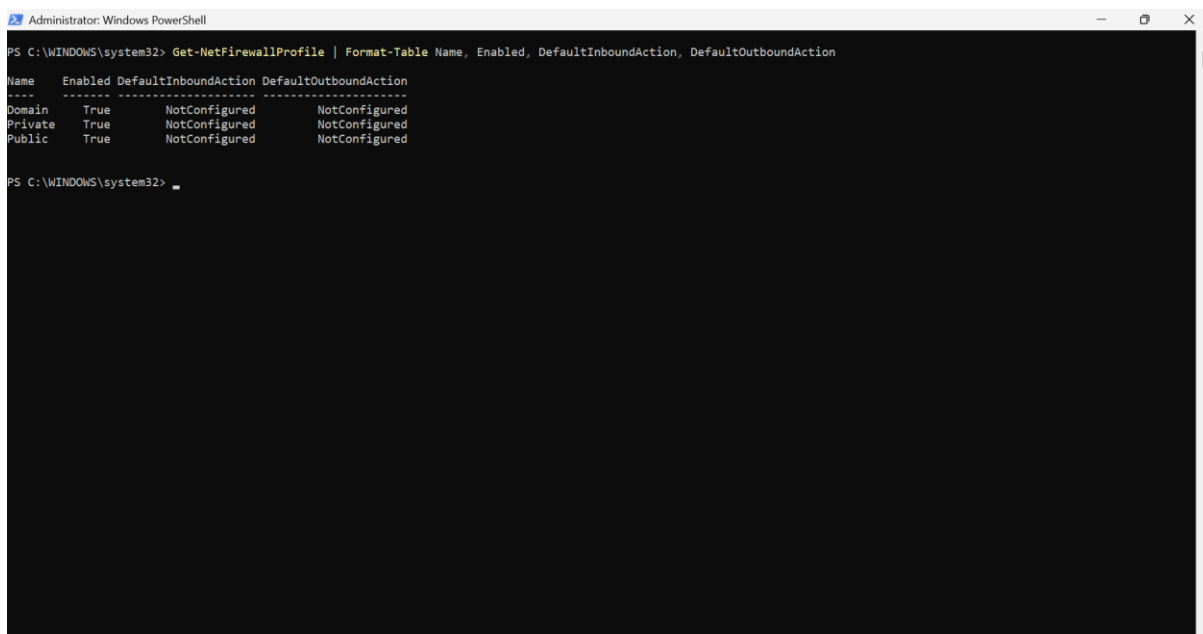
1. Baseline system assessment.
 2. Verification of existing firewall configuration.
 3. Creation of custom Windows Defender Firewall rules.
 4. Configuration of Windows security settings.
 5. System hardening using built-in Windows security features.
 6. Verification using PowerShell and Command Prompt.
 7. Documentation of results and evidence collection.
-

Phase	Activity	Outcome
Phase 1	Baseline Assessment	System information collected
Phase 2	Firewall Verification	Existing firewall status confirmed
Phase 3	Firewall Rule Configuration	Custom rules created
Phase 4	Windows Hardening	Security features verified
Phase 5	PowerShell Verification	Rules validated
Phase 6	Security Testing	Configuration confirmed
Phase 7	Documentation	Technical report prepared

Table 4.1: Project Implementation Methodology

4.2 Baseline Firewall Assessment

Before creating new firewall rules, the existing Windows Defender Firewall configuration was reviewed to verify that all firewall profiles were enabled and operating correctly. Windows Security and Windows Defender Firewall with Advanced Security were used to confirm the status of the Domain, Private, and public firewall profiles. The baseline assessment confirmed that Windows Defender Firewall was enabled on all available network profiles, providing a secure foundation for implementing additional custom firewall rules.



```

Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Get-NetFirewallProfile | Format-Table Name, Enabled, DefaultInboundAction, DefaultOutboundAction
Name      Enabled DefaultInboundAction DefaultOutboundAction
-----
Domain    True    NotConfigured         NotConfigured
Private   True    NotConfigured         NotConfigured
Public    True    NotConfigured         NotConfigured
PS C:\WINDOWS\system32>
  
```

Figure 4.2 – Windows Defender Firewall status

Windows Firewall Configuration & System Hardening

```
Administrator: Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> Get-NetFirewallProfile

Name                : Domain
Enabled              : True
DefaultInboundAction : NotConfigured
DefaultOutboundAction : NotConfigured
AllowInboundRules    : NotConfigured
AllowLocalFirewallRules : NotConfigured
AllowLocalIPsecRules : NotConfigured
AllowUserApps        : NotConfigured
AllowUserPorts       : NotConfigured
AllowUnicastResponseToMulticast : NotConfigured
NotifyOnListen       : True
EnableStealthModeForIPsec : NotConfigured
LogFileName          : %systemroot%\system32\LogFiles\Firewall\pfirewall.log
LogMaxSizeKilobytes  : 4096
LogAllowed            : False
LogBlocked            : False
LogIgnored            : NotConfigured
DisabledInterfaceAliases : {NotConfigured}

Name                : Private
Enabled              : True
DefaultInboundAction : NotConfigured
DefaultOutboundAction : NotConfigured
AllowInboundRules    : NotConfigured
AllowLocalFirewallRules : NotConfigured
AllowLocalIPsecRules : NotConfigured
AllowUserApps        : NotConfigured
AllowUserPorts       : NotConfigured
AllowUnicastResponseToMulticast : NotConfigured
NotifyOnListen       : True
EnableStealthModeForIPsec : NotConfigured
LogFileName          : %systemroot%\system32\LogFiles\Firewall\pfirewall.log
LogMaxSizeKilobytes  : 4096
LogAllowed            : False
```

Figure 4.2: Windows Defender Firewall Profile Status Before Configuration

Explanation:

Figure 4.2 illustrates the initial Windows Defender Firewall profile configuration before any custom firewall rules were created. The Domain, Private, and public firewall profiles were confirmed to be enabled, ensuring that the operating system was protected before implementing additional security controls.

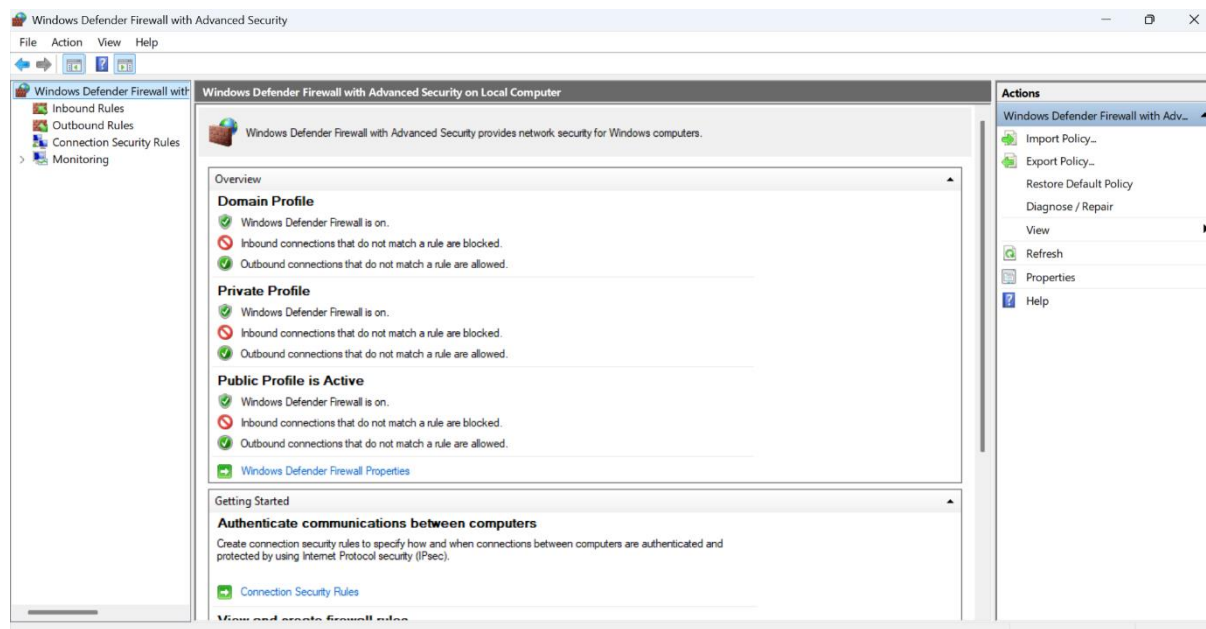


Figure 4.3: Windows Defender Firewall with Advanced Security Console

Explanation:

The Windows Defender Firewall with Advanced Security console provides centralized management of inbound rules, outbound rules, connection security rules, and monitoring. This interface was used throughout the project to create and manage custom firewall rules.

Firewall Profile	Status
Domain	Enabled
Private	Enabled
Public	Enabled
Overall Firewall Status	Active

Figure 4.3: Windows Defender Firewall with Advanced Security Console

4.3 Custom Firewall Rule Configuration

After verifying the default firewall configuration, custom inbound firewall rules were created using **Windows Defender Firewall with Advanced Security**. The objective was to allow only essential network services while blocking commonly exploited insecure services. Each rule was created manually and verified after implementation to ensure the desired firewall behavior. The selected ports represent common enterprise scenarios where secure communication is permitted while insecure legacy protocols are restricted. All rules were applied to the **Domain**, **Private**, and **Public** firewall profiles to provide consistent protection across different network environments.

4.3.1 HTTP Allow Rule (Port 80)

The first custom firewall rule was created to allow inbound HTTP traffic on **TCP Port 80**. Although the workstation is not acting as a web server, this rule demonstrates the ability to configure inbound rules for web-based services commonly used in enterprise environments.

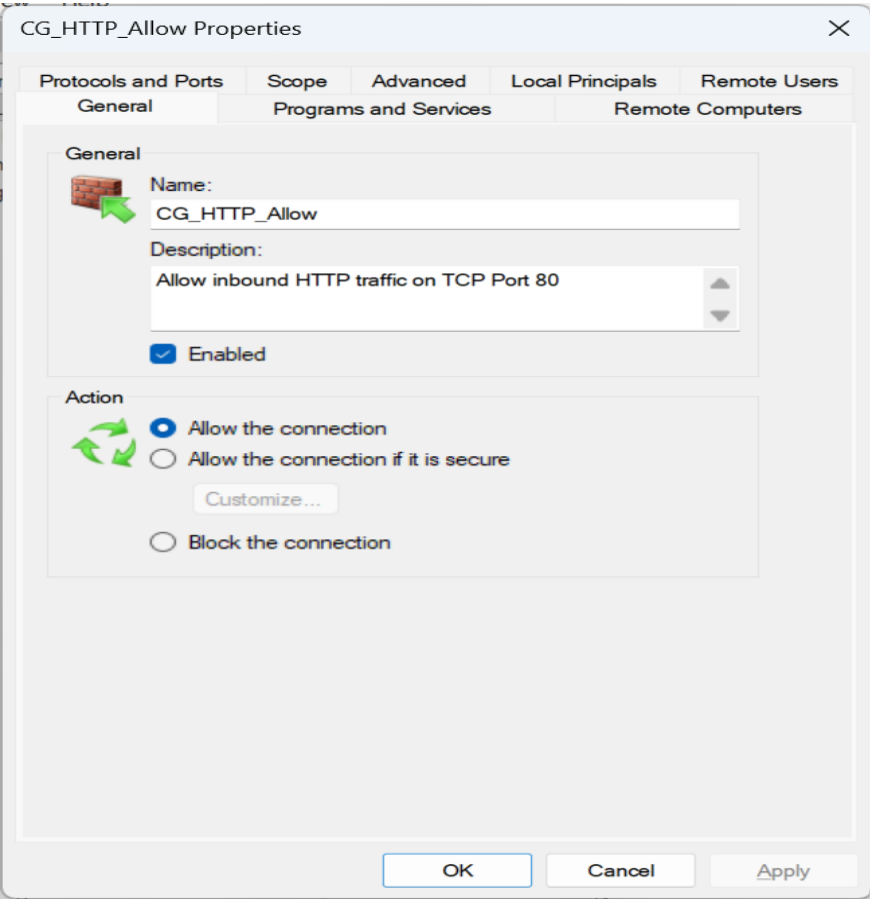


Figure 4.4: Custom Firewall Rule – HTTP Allow (TCP Port 80)

Explanation

Figure 4.4 illustrates the custom inbound firewall rule created to allow HTTP traffic over TCP Port 80. The rule was configured for all Windows Firewall profiles and demonstrates secure firewall administration practices using Windows Defender Firewall with Advanced Security.

Parameter	Value
Rule Name	CG_HTTP_Allow
Direction	Inbound
Protocol	TCP
Port	80
Action	Allow

Table 4.3: HTTP Firewall Rule Configuration

4.3.2 HTTPS Allow Rule (Port 443)

HTTPS is the standard protocol for secure web communication. A custom inbound rule was created to allow encrypted web traffic over TCP Port 443.

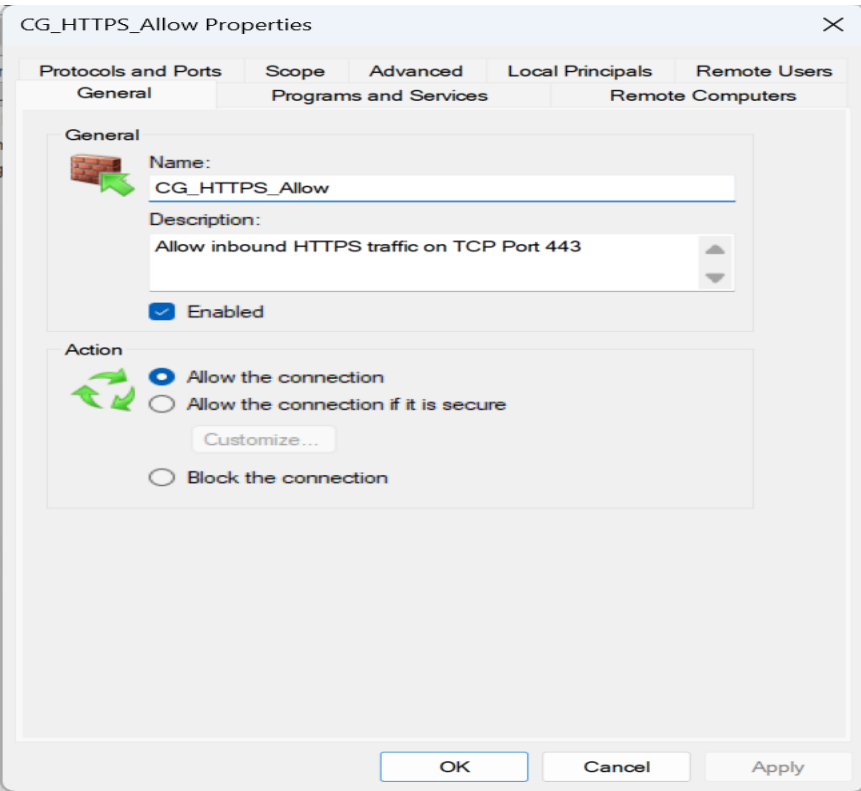


Figure 4.5: Custom Firewall Rule – HTTPS Allow (TCP Port 443)

Explanation : This firewall rule permits inbound HTTPS communication using TCP Port 443. HTTPS encrypts network traffic using TLS, ensuring secure communication between clients and web services.

Parameter	Value
Rule Name	CG_HTTPS_Allow
Direction	Inbound
Protocol	TCP
Port	443
Action	Allow
Profiles	Domain, Private, Public

Table 4.4 – HTTPS Rule Configuration

4.3.3 SSH Allow Rule (Port 22)

Secure Shell (SSH) is widely used for secure remote administration of systems. A custom rule was created to allow inbound SSH communication.

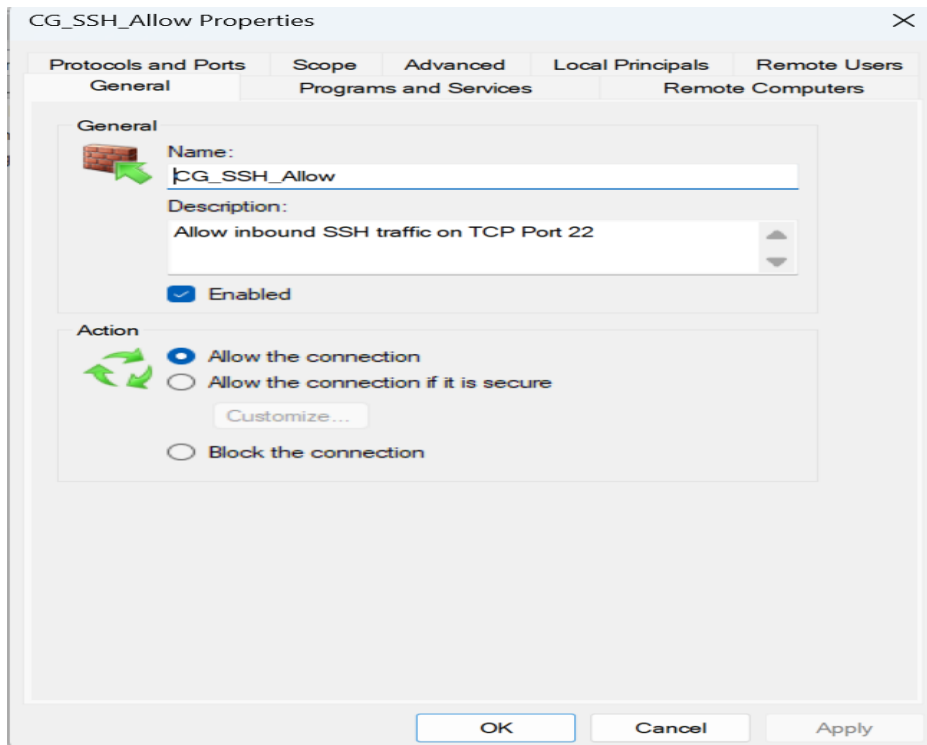


Figure 4.6: Custom Firewall Rule – SSH Allow (TCP Port 22)

Explanation

The SSH rule demonstrates the configuration of secure remote administration services. Unlike Telnet, SSH encrypts communication, making it suitable for secure remote management.

Parameter	Value
Rule Name	CG_SSH_Allow
Direction	Inbound
Protocol	TCP
Port	22
Action	Allow
Profiles	Domain, Private, Public

Table 4.5 – SSH Rule Configuration

4.3.4 Firewall Rule Summary

After configuring the allow rules, the firewall was reviewed to verify that all newly created rules were active and correctly configured.

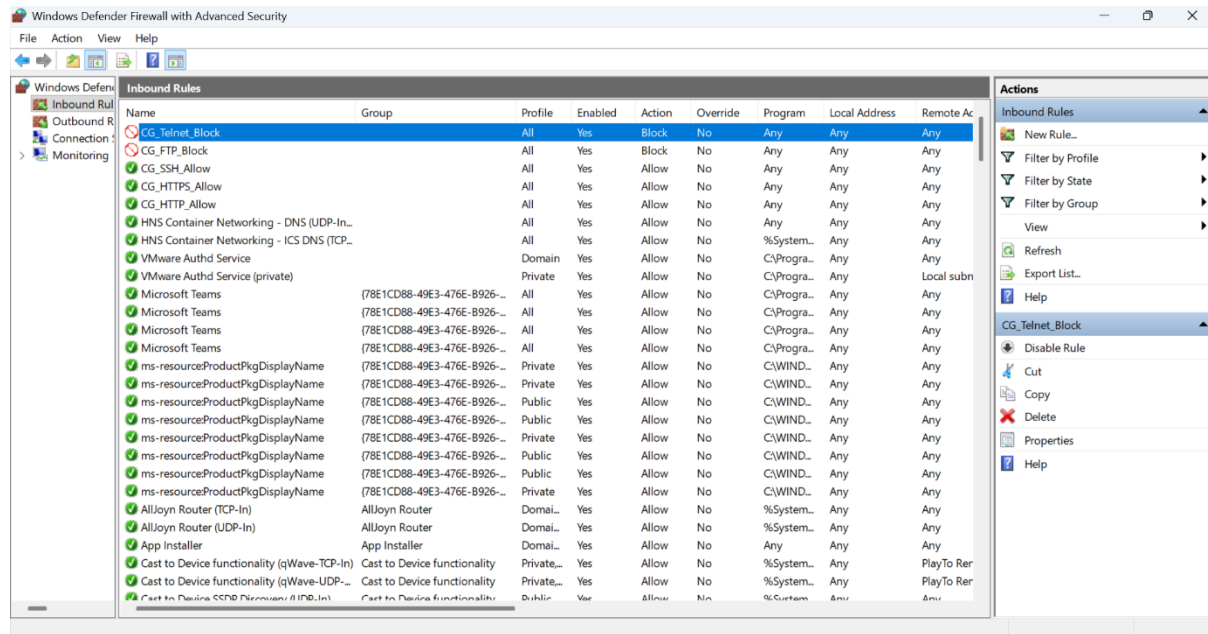


Figure 4.7: Custom Firewall Rules Created During the Project

Explanation

Figure 4.7 displays the list of custom firewall rules created during the project. The rules demonstrate controlled access by allowing secure protocols while preparing for the implementation of blocking rules for insecure services.

Rule Name	Port	Protocol	Action	Purpose
CG_HTTP_Allow	80	TCP	Allow	Web Traffic
CG_HTTPS_Allow	443	TCP	Allow	Secure Web Traffic
CG_SSH_Allow	22	TCP	Allow	Secure Remote Access

4.4 Blocking Insecure Network Services

While secure services are required for normal business operations, several legacy network protocols continue to pose significant security risks because they transmit information without encryption. Two commonly exploited protocols are File Transfer Protocol (FTP) and Telnet. These protocols transmit credentials and network traffic in plaintext, making them vulnerable to packet sniffing, credential theft, and man-in-the-middle attacks.

To reduce the attack surface of the Windows workstation, custom firewall rules were created to block inbound traffic on TCP Port 21 (FTP) and TCP Port 23 (Telnet). Blocking these ports follows modern cybersecurity best practices and aligns with recommendations from Microsoft and the Center for Internet Security (CIS).

4.4.1 FTP Block Rule (Port 21)

File Transfer Protocol (FTP) was one of the earliest file transfer protocols developed for computer networks. However, because FTP does not encrypt authentication credentials or transmitted data, it is considered insecure for modern enterprise environments.

A custom firewall rule named **CG_FTP_Block** was created to prevent inbound FTP communication over TCP Port 21.

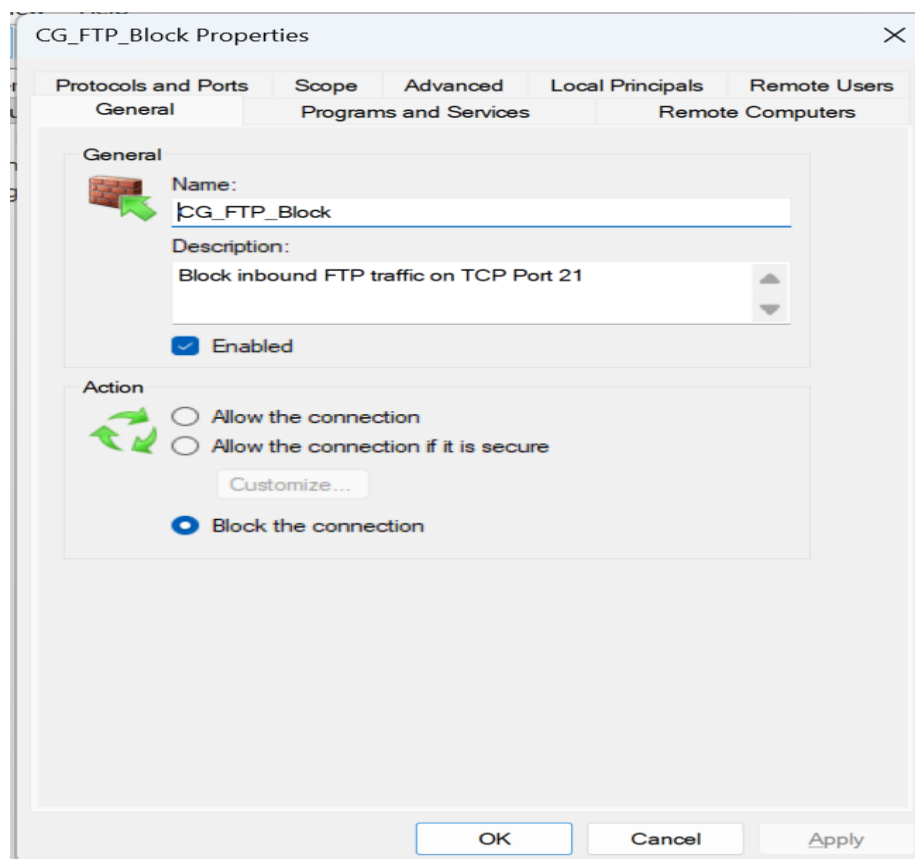


Figure 4.9: Custom Firewall Rule – Block FTP (TCP Port 21)

Explanation

Figure 4.9 illustrates the firewall rule configured to block inbound FTP traffic on TCP Port 21. Blocking FTP prevents the use of insecure file transfer services that expose authentication credentials and sensitive information over unencrypted network connections.

Parameter	Value
Rule Name	CG_FTP_Block
Direction	Inbound
Protocol	TCP
Port	21
Action	Block
Profiles	Domain, Private, Public

Table 4.7 – FTP Rule Configuration

4.4.2 Telnet Block Rule (Port 23)

Telnet is a legacy remote administration protocol that transmits user credentials in plaintext. Because Telnet does not provide encryption, it is highly susceptible to credential interception and network attacks.

To eliminate this security risk, a custom firewall rule named **CG_Telnet_Block** was created to block inbound Telnet traffic.

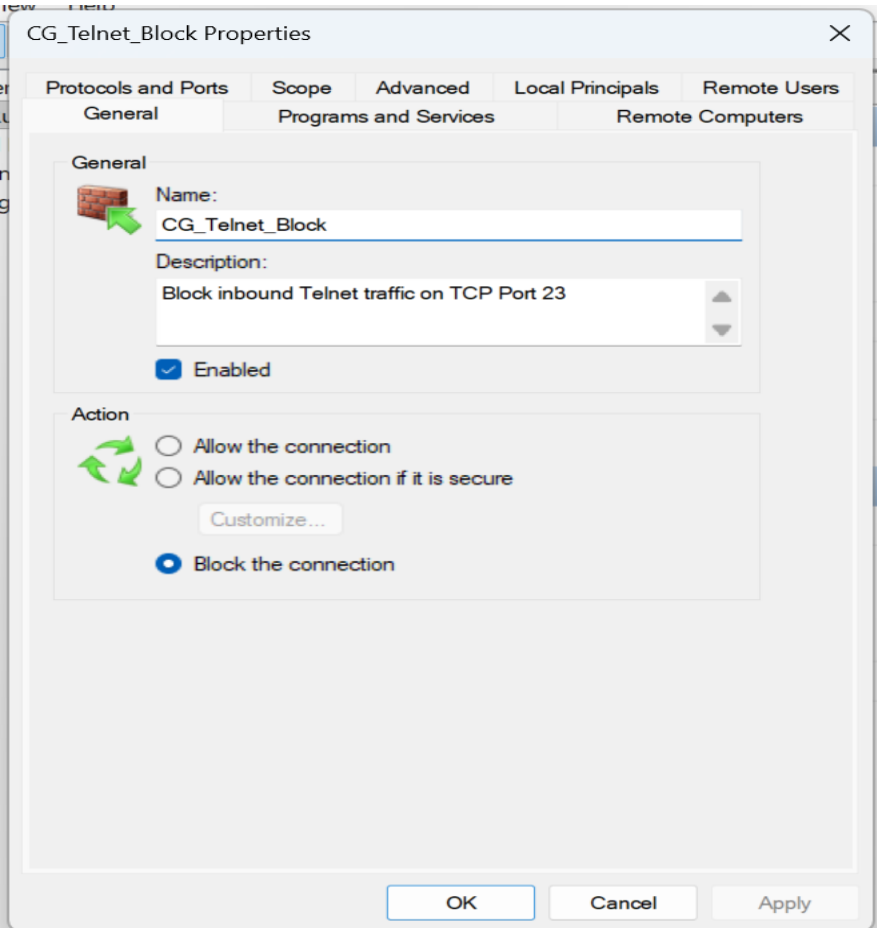


Figure 4.10: Custom Firewall Rule – Block Telnet (TCP Port 23)

Explanation

Figure 4.10 shows the firewall rule created to block inbound Telnet communication. This configuration prevents insecure remote administration and encourages the use of encrypted protocols such as SSH.

Parameter	Value
Rule Name	CG_Telnet_Block
Direction	Inbound
Protocol	TCP
Port	23
Action	Block
Profiles	Domain, Private, Public

Table 4.8 – Telnet Rule Configuration

4.5 Complete Firewall Rule Summary

After configuring all custom firewall rules, the Windows Defender Firewall configuration consisted of three allow rules and two block rules. The firewall policy followed the principle of **least privilege**, allowing only required services while denying insecure protocols.

Rule Name	Port	Protocol	Action	Security Purpose
CG_HTTP-Allow	80	TCP	Allow	Allow HTTP traffic
CG_HTTPS-Allow	443	TCP	Allow	Allow encrypted web traffic
CG_SSH-Allow	22	TCP	Allow	Secure remote administration
CG_FTP-Block	21	TCP	Block	Prevent insecure file transfer
CG_Telnet-Block	23	TCP	Block	Prevent insecure remote access

Table 4.9 – Complete Firewall Rule Matrix

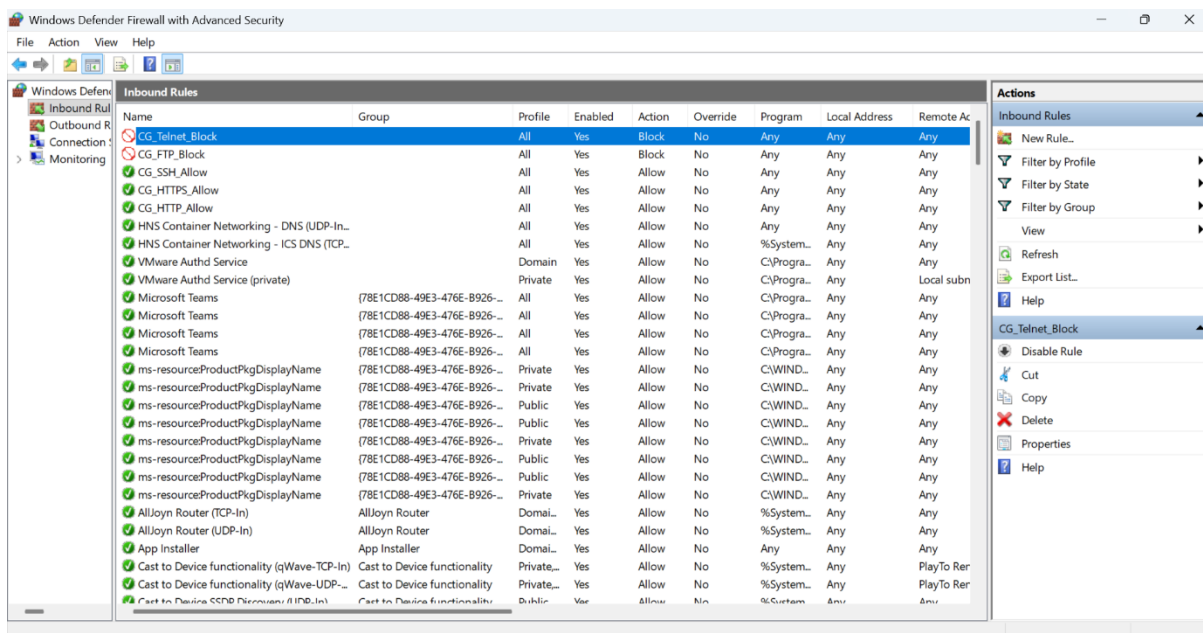


Figure 4.11: Final Custom Firewall Rules Configured During the Project

Explanation

Figure 4.11 presents the complete set of custom firewall rules implemented during the project. The configuration demonstrates a balanced security approach by permitting secure network services while restricting insecure legacy protocols. This rule set reduces the overall attack surface of the endpoint without affecting essential system functionality.

Chapter Summary

This section documented the implementation of custom Windows Defender Firewall rules. Secure services including HTTP, HTTPS, and SSH were explicitly allowed, while insecure services such as FTP and Telnet were blocked. The resulting firewall policy follows the principle of least privilege and reduces the attack surface of the Windows workstation.

4.6 Firewall Verification and Validation

After configuring the custom firewall rules, a comprehensive verification process was performed to ensure that all configurations had been successfully applied. Validation was carried out using both graphical and command-line administrative tools available in Microsoft Windows.

Windows Command Prompt and Windows PowerShell were used to verify the presence, status, and operational state of each custom firewall rule. This validation process ensured that the configured rules were enabled, correctly associated with their respective ports, and actively enforced by Windows Defender Firewall.

The verification process also confirmed that all three firewall profiles (Domain, Private, and Public) remained enabled after implementing the custom rule set.

4.6.1 Firewall Rule Verification using Command Prompt

Windows Command Prompt provides the **netsh advfirewall** utility, which enables administrators to display and manage Windows Defender Firewall rules through the command-line interface.

The following command was executed:

```
netsh advfirewall firewall show rule name=all
```

The output displayed all configured firewall rules, including the custom rules created during this project.

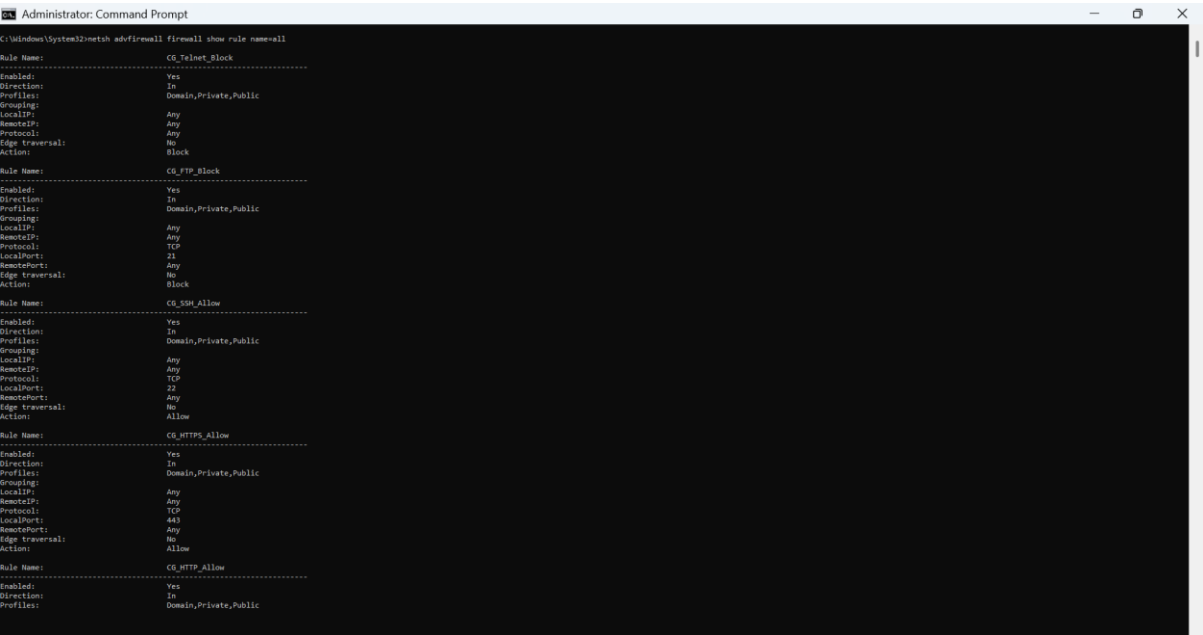


Figure 4.13: Firewall Rule Verification using netsh

Explanation

Figure 4.13 displays the output generated by the **netsh advfirewall firewall show rule name=all** command. The output confirms that all custom firewall rules were successfully registered within Windows Defender Firewall and are available for enforcement.

Verification Item	Status
Firewall Rules Listed	Successful
HTTP Rule Found	Yes
HTTPS Rule Found	Yes
SSH Rule Found	Yes
FTP Block Rule Found	Yes
Telnet Block Rule Found	Yes

Table 4.10 – Command Prompt Verification

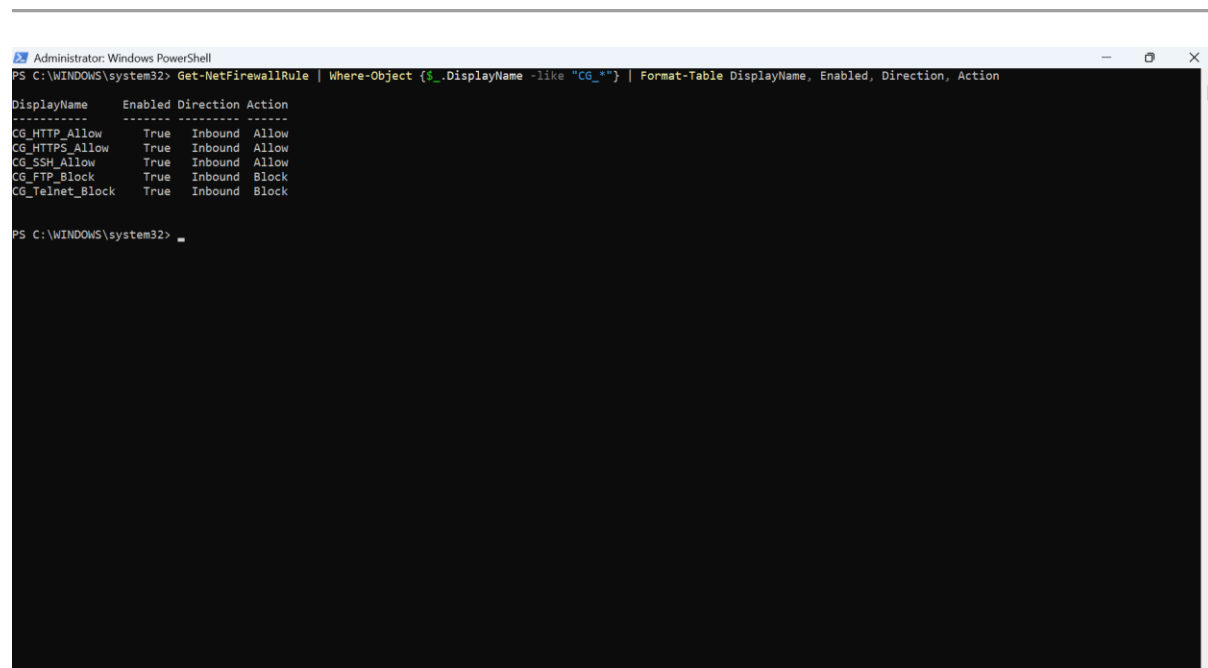
4.6.2 Firewall Verification using Windows PowerShell

Windows PowerShell provides advanced firewall management cmdlets that allow administrators to validate firewall rules programmatically.

The following PowerShell command was executed:

```
Get-NetFirewallRule |  
Where-Object {$_.DisplayName -like "CG_*"} |  
Format-Table DisplayName, Enabled, Direction, Action
```

The command filtered only the firewall rules created during this project.



```
Administrator: Windows PowerShell  
PS C:\WINDOWS\system32> Get-NetFirewallRule | Where-Object {$_.DisplayName -like "CG_*"} | Format-Table DisplayName, Enabled, Direction, Action  
-----  
DisplayName      Enabled Direction Action  
-----  
CG_HTTP_Allow    True    Inbound  Allow  
CG_HTTPS_Allow   True    Inbound  Allow  
CG_SSH_Allow     True    Inbound  Allow  
CG_FTP_Block     True    Inbound  Block  
CG_Telnet_Block  True    Inbound  Block  
  
PS C:\WINDOWS\system32>
```

Figure 4.14: PowerShell Verification of Custom Firewall Rules

Explanation

Figure 4.14 confirms that all custom firewall rules are enabled and correctly configured. The PowerShell output verifies the display name, rule direction, and action associated with each firewall rule, demonstrating successful implementation.

Rule	Enabled	Direction	Action
CG_HTTP_Allow	True	Inbound	Allow
CG_HTTPS_Allow	True	Inbound	Allow
CG_SSH_Allow	True	Inbound	Allow
CG_FTP_Block	True	Inbound	Block
CG_Telnet_Block	True	Inbound	Block

Table 4.11 – PowerShell Verification Results

4.6.3 Port Filter Verification

To validate that the firewall rules were correctly associated with their intended TCP ports, Windows PowerShell was used to inspect the configured firewall port filters.

The following command was executed:

```
Get-NetFirewallPortFilter |
Where-Object {$_.LocalPort -in 21,22,23,80,443} |
Format-Table Protocol, LocalPort
```

```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Get-NetFirewallRule | Where-Object {$_.DisplayName -like "CG_*"} | Format-Table DisplayName, Enabled, Direction, Action
-----
DisplayName      Enabled Direction Action
-----
CG_HTTP_Allow    True    Inbound  Allow
CG_HTTPS_Allow   True    Inbound  Allow
CG_SSH_Allow     True    Inbound  Allow
CG_FTP_Block     True    Inbound  Block
CG_Telnet_Block  True    Inbound  Block

PS C:\WINDOWS\system32> Get-NetFirewallPortFilter |
>> Where-Object {$_.LocalPort -in 21,22,23,80,443} |
>> Format-Table Protocol, LocalPort
-----
Protocol LocalPort
-----
TCP      80
TCP      443
TCP      80
TCP      80
TCP      443
TCP      22
TCP      21

PS C:\WINDOWS\system32>
```

Figure 4.15 – Port Filter Verification

Explanation

The PowerShell output confirms that Windows Defender Firewall correctly associates the configured firewall rules with TCP ports **21**, **22**, **23**, **80**, and **443**. This verification ensures that the implemented firewall policy is applied to the intended network services.

Port	Protocol	Verification Status
21	TCP	Verified
22	TCP	Verified
23	TCP	Verified
80	TCP	Verified
443	TCP	Verified

Table 4.12 – Port Verification

4.6.4 Firewall Profile Verification

Following the implementation and verification of custom firewall rules, the operational status of the Windows Defender Firewall profiles was validated.

PowerShell confirmed that the Domain, Private, and Public firewall profiles remained enabled, ensuring that firewall protection continued to be enforced across all network environments.

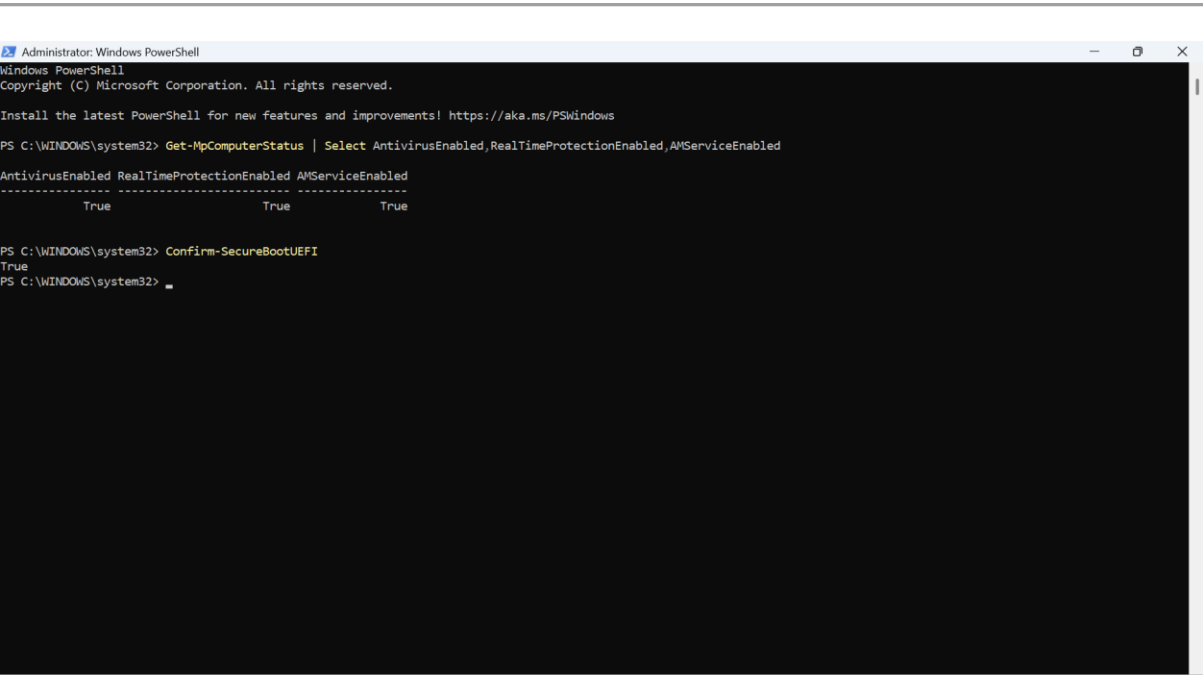


Figure 4.16: Windows Defender Firewall Profile Verification

Explanation

Figure 4.16 verifies that all Windows Defender Firewall profiles remain enabled after implementing the firewall configuration. Maintaining active firewall protection across Domain, Private, and Public profiles is essential for consistent endpoint security.

Firewall Profile	Status
Domain	Enabled
Private	Enabled
Public	Enabled

Table 4.13 – Firewall Profile Status

Verification Area	Result
Firewall Rule Creation	Successful
Rule Verification	Successful
Port Validation	Successful
Firewall Profiles	Active
Overall Configuration	Successfully Implemented

Table 4.14: Firewall Verification Summary

4.7 Windows System Hardening Implementation

Following the successful configuration and verification of Windows Defender Firewall, the operating system was further secured through the implementation and verification of multiple Windows security features. Windows 11 provides several built-in mechanisms that protect against malware, unauthorized access, credential theft, and kernel-level attacks. These security controls work together to provide a layered defense model that significantly improves endpoint security.

The hardening process included verification of Microsoft Defender Antivirus, Windows Security Dashboard, Device Security, Secure Boot, Trusted Platform Module (TPM), Core Isolation, Memory Integrity, Windows Update, Device Encryption, User Account Control (UAC), and Guest Account status.

4.7.1 Windows Security Dashboard

The Windows Security Dashboard provides a centralized interface for monitoring the security status of the operating system. It displays the operational status of Microsoft Defender Antivirus, Firewall & Network Protection, Device Security, App & Browser Control, Account Protection, and Device Performance & Health.

Reviewing this dashboard provides a quick assessment of the overall security posture before and after implementing security hardening measures.

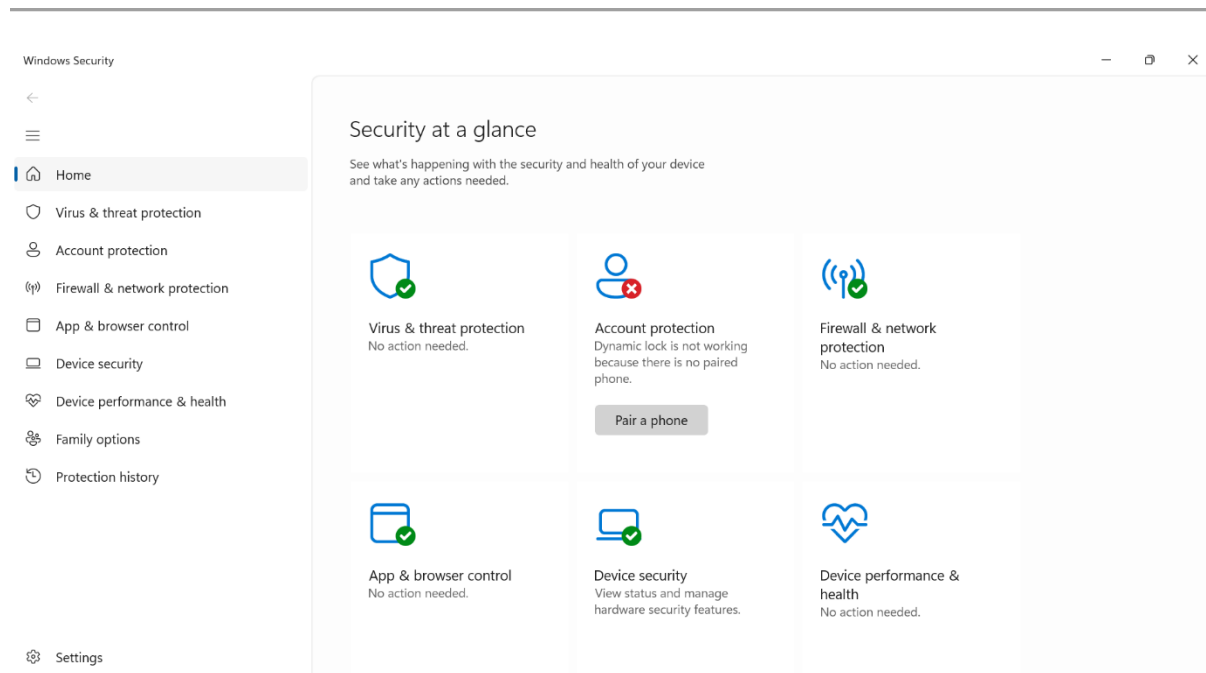


Figure 4.17: Windows Security Dashboard

Explanation

Figure 4.17 displays the Windows Security Dashboard showing the overall health status of the operating system. All major security components were operational, indicating that the Windows security environment was functioning correctly before detailed verification of individual security features.

4.7.2 Microsoft Defender Antivirus

Microsoft Defender Antivirus provides real-time protection against malware, ransomware, spyware, and other malicious software. During the implementation process, the antivirus configuration was reviewed to verify that malware protection services were active.

The Virus & Threat Protection section confirmed that the system had no active threats and that Microsoft Defender was successfully protecting the operating system.

Windows Firewall Configuration & System Hardening

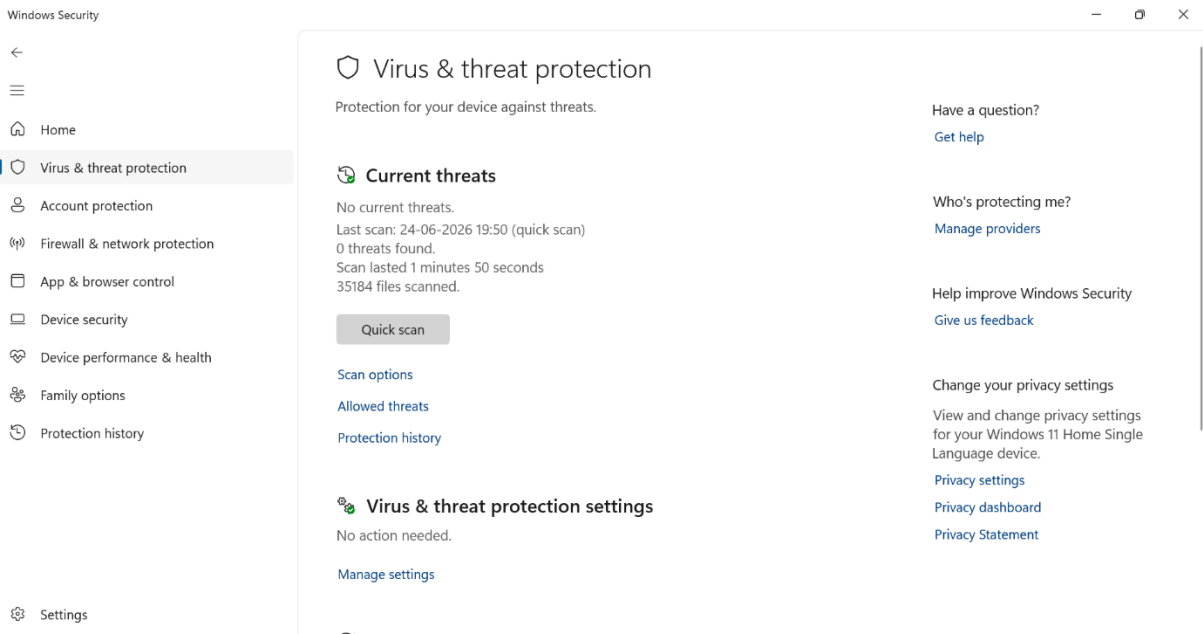


Figure 4.18: Microsoft Defender Virus & Threat Protection

Explanation

Figure 4.18 confirms that Microsoft Defender Antivirus detected no active threats during the assessment. The successful completion of the latest security scan demonstrates that the endpoint remained protected throughout the implementation process.

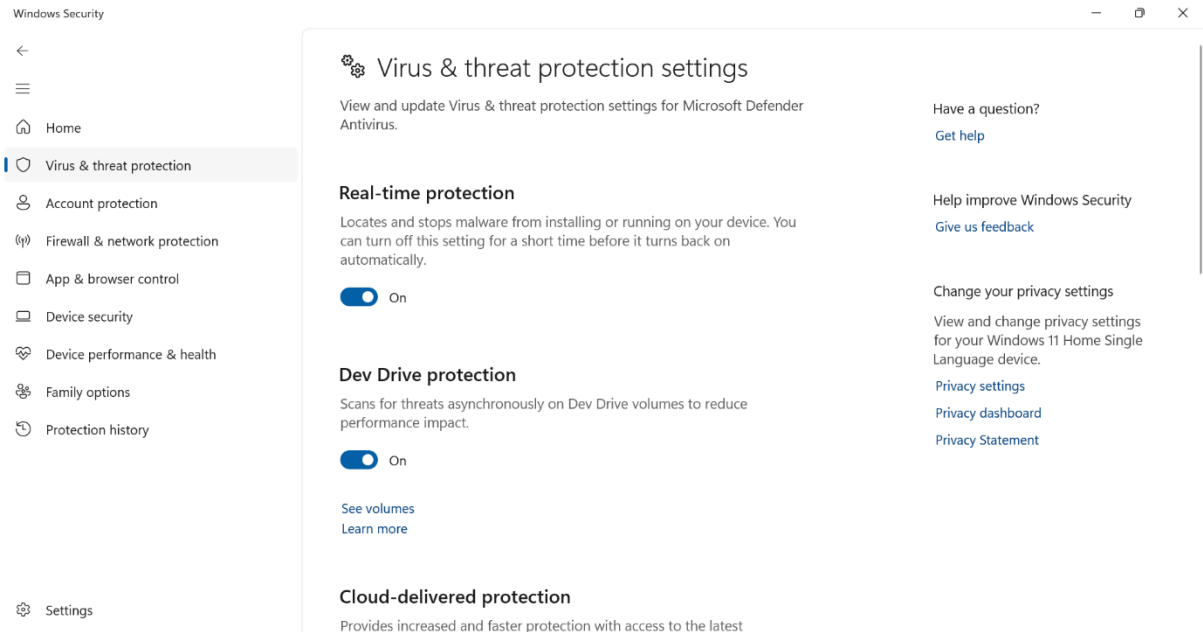


Figure 4.19: Microsoft Defender Security Settings

Explanation

The Defender configuration settings verify that essential protection mechanisms including Real-Time Protection, Cloud-Delivered Protection, Automatic Sample Submission, and Tamper Protection were enabled. These features provide continuous protection against known and emerging threats.

Security Feature	Status
Real-Time Protection	Enabled
Cloud-Delivered Protection	Enabled
Automatic Sample Submission	Enabled
Tamper Protection	Enabled
Antivirus Engine	Active

Table 4.15 – Microsoft Defender Configuration

4.7.3 Device Security

Windows Device Security provides hardware-backed protection using Secure Boot, Trusted Platform Module (TPM), virtualization-based security, and hardware isolation technologies. These security features help protect the operating system from firmware attacks, credential theft, and unauthorized modifications during the boot process.

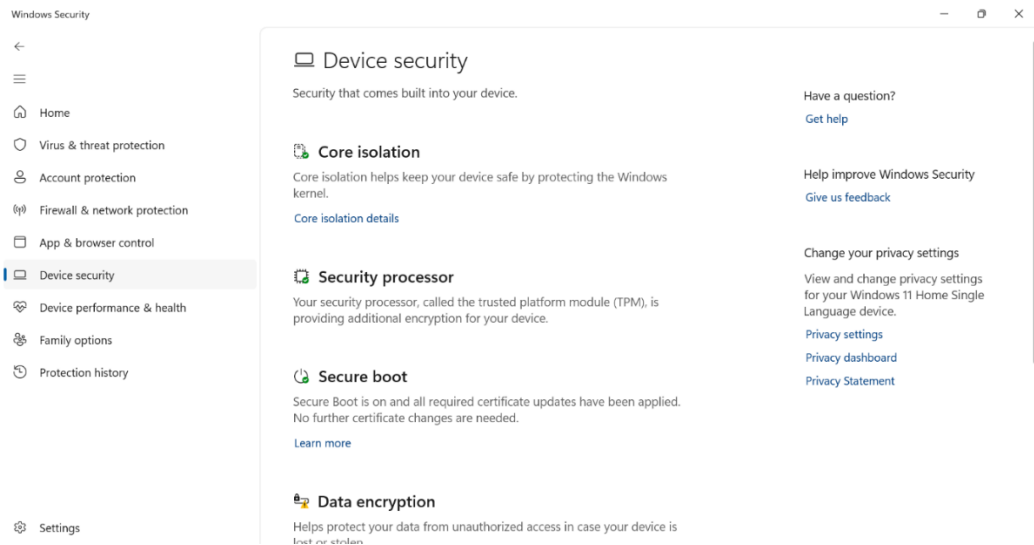


Figure 4.20: Windows Device Security

Explanation

Figure 4.20 illustrates the Device Security interface confirming that Secure Boot, TPM, and other hardware-based security mechanisms were available and operational on the assessment system.

Feature	Status	Security Benefit
Secure Boot	Enabled	Prevents unauthorized boot code
TPM	Available	Hardware-backed security
Hardware Security	Supported	Improved system integrity
Virtualization-Based Security	Enabled	Kernel protection

Table 4.16 – Device Security Features

4.7.4 Core Isolation and Memory Integrity

Core Isolation is a virtualization-based security feature that isolates critical operating system processes from the rest of the system. Memory Integrity, also known as Hypervisor-Protected Code Integrity (HVCI), prevents malicious code and unauthorized drivers from modifying protected areas of memory.

During this project, the Core Isolation settings were examined to verify that Memory Integrity and Local Security Authority (LSA) Protection were enabled. These security mechanisms strengthen Windows kernel security and provide additional protection against advanced malware and credential theft attacks.

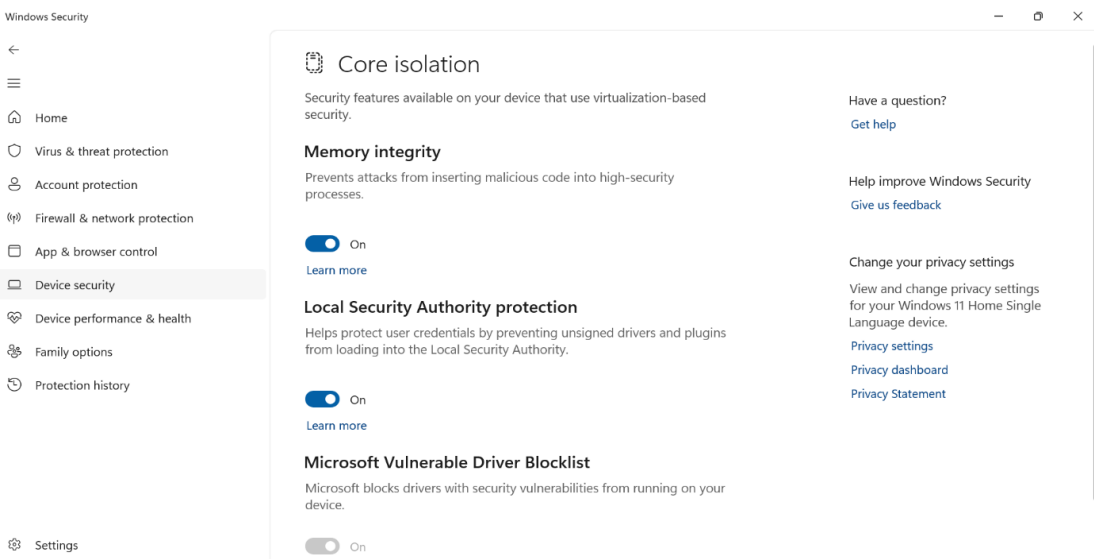


Figure 4.21: Windows Core Isolation Security Settings

Explanation

Figure 4.21 illustrates the Core Isolation configuration. Memory Integrity and Local Security Authority (LSA) Protection were enabled, providing additional protection against malicious drivers, kernel-level attacks, and credential theft.

Security Feature	Status	Purpose
Memory Integrity	Enabled	Prevent malicious kernel code
LSA Protection	Enabled	Protect Windows credentials
Vulnerable Driver Blocklist	Disabled	Optional compatibility feature

Table 4.17 – Core Isolation Configuration

4.7.5 Windows Update

Keeping the operating system updated is one of the most important cybersecurity best practices. Windows Update provides security patches, bug fixes, and feature improvements that help protect the operating system against newly discovered vulnerabilities.

The Windows Update service was verified during the implementation process to ensure that update functionality was operational.

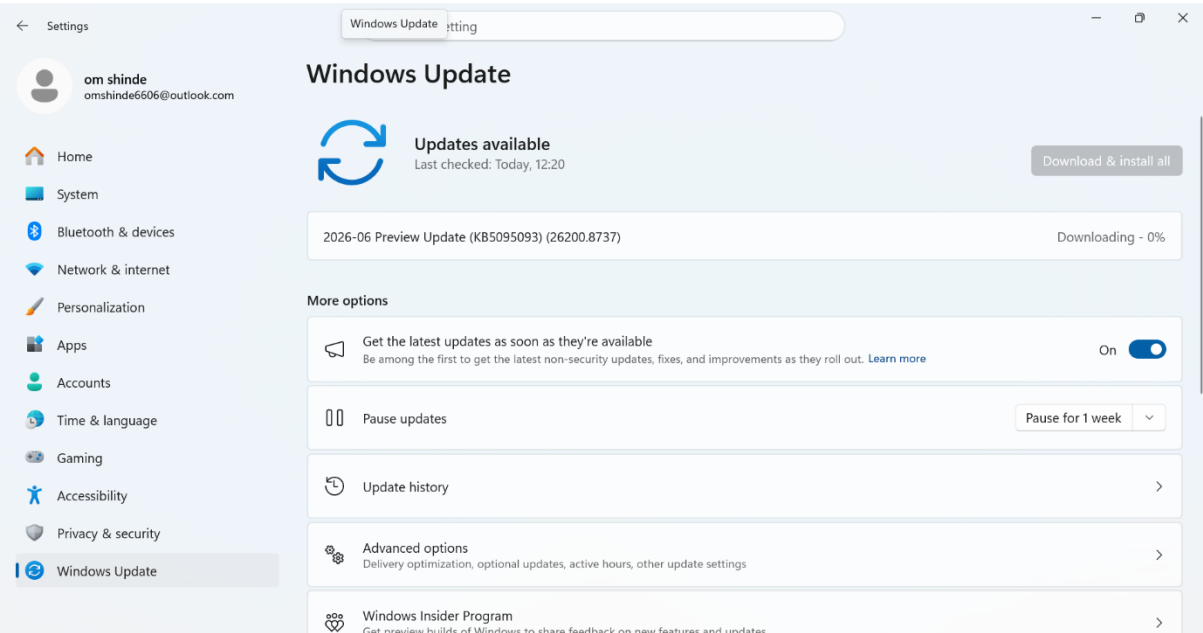


Figure 4.22: Windows Update Verification

Explanation

Figure 4.22 confirms that Windows Update was operational and capable of downloading the latest security updates. Maintaining an updated operating system significantly reduces exposure to known vulnerabilities.

4.7.6 Device Encryption

Device Encryption protects sensitive information stored on the system by encrypting the operating system drive. Even if the storage device is removed or the system is lost, encrypted data cannot be accessed without proper authentication.

The project verified that Device Encryption was enabled, demonstrating compliance with modern endpoint security recommendations.

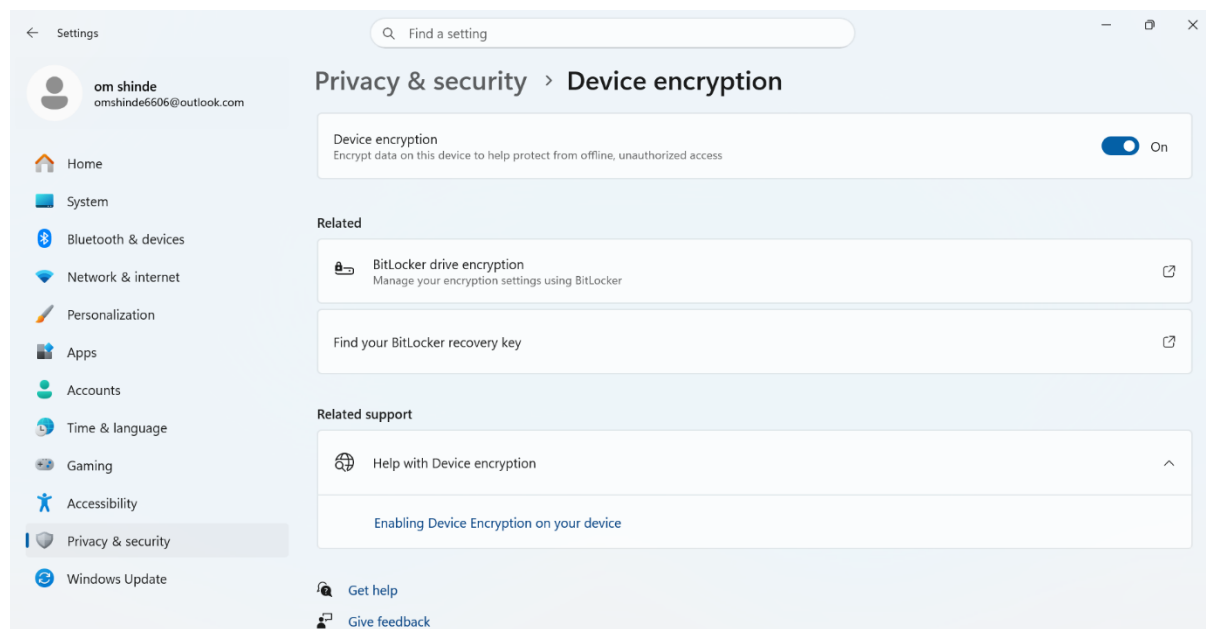


Figure 4.23: Device Encryption Status

Explanation

Figure 4.23 verifies that Device Encryption was enabled on the assessment system. Encryption protects stored information against unauthorized access and supports data confidentiality.

Feature	Status	Benefit
Device Encryption	Enabled	Protects stored data
Recovery Key Support	Available	Disaster Recovery
Secure Boot Integration	Enabled	Secure startup

Table 4.18 – Data Protection Features

4.7.7 User Account Control (UAC)

User Account Control restricts administrative privileges and prevents unauthorized software from making changes to the operating system without user approval.

During the assessment, UAC was configured at the Microsoft-recommended security level.c

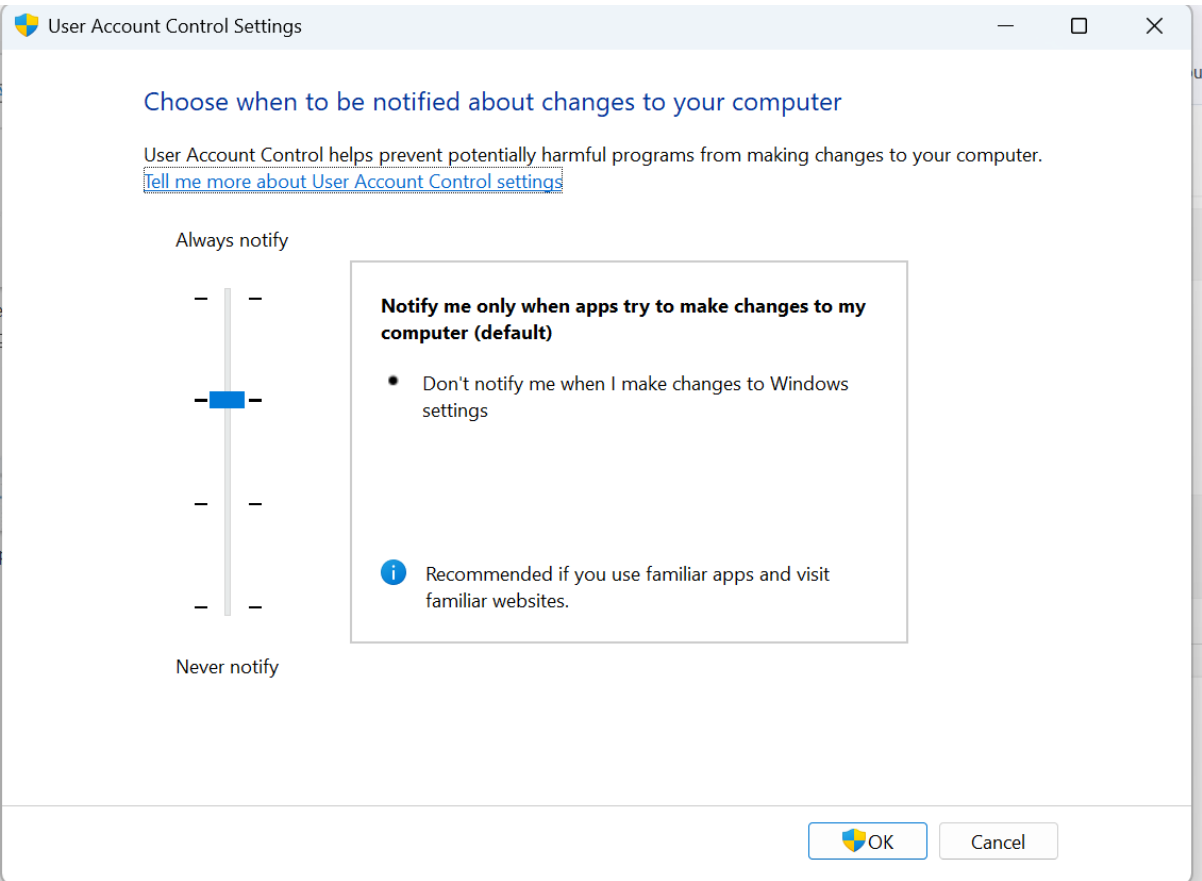


Figure 4.24: User Account Control Configuration

Explanation

The User Account Control configuration was verified to ensure that administrative actions require user consent before execution. This reduces the risk of unauthorized system modifications.

4.7.8 Guest Account Verification

The built-in Guest account provides limited access to Windows systems. From a security perspective, this account should remain disabled unless explicitly required.

The Guest account status was verified using Command Prompt.

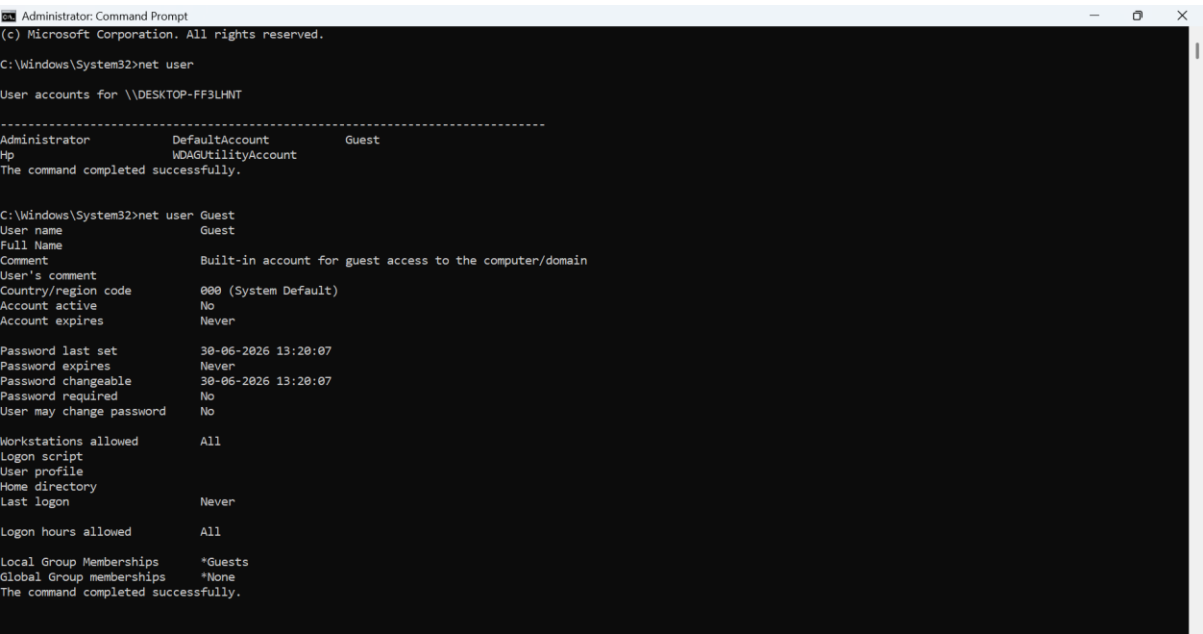


Figure 4.25: Guest Account Status Verification

Explanation

Figure 4.25 confirms that the Windows Guest account was disabled. Disabling unused accounts reduces the attack surface and prevents unauthorized access.

Security Control	Status
Windows Defender Antivirus	Enabled
Firewall	Enabled
Secure Boot	Enabled

Security Control	Status
TPM	Available
Memory Integrity	Enabled
Device Encryption	Enabled
Windows Update	Operational
UAC	Enabled
Guest Account	Disabled

Table 4.19 – Windows Hardening Summary

Chapter Summary

This chapter presented the complete implementation of Windows Defender Firewall configuration and Windows 11 system hardening. Firewall rules were created and verified using PowerShell and Command Prompt, while built-in Windows security features including Microsoft Defender Antivirus, Secure Boot, TPM, Core Isolation, Device Encryption, Windows Update, User Account Control, and Guest Account settings were successfully verified. The implementation demonstrates a layered security approach that strengthens endpoint protection and reduces the overall attack surface of the operating system.

CHAPTER 5

5. RESULTS, ANALYSIS AND SECURITY ASSESSMENT

The successful implementation of Windows Defender Firewall configuration and Windows system hardening significantly improved the security posture of the assessment system. All planned firewall rules were successfully created, verified, and enforced using Windows Defender Firewall. Additionally, multiple built-in Windows security mechanisms were validated to ensure comprehensive endpoint protection.

The results obtained during the implementation demonstrate that Windows provides a robust set of native security features capable of protecting enterprise endpoints against unauthorized network access, malware, credential theft, and operating system compromise. Verification using Windows PowerShell, Command Prompt, and Windows Security confirmed that all implemented security controls were functioning as intended.

This chapter presents the security improvements achieved during the project together with an analysis of the implemented firewall policy and Windows hardening measures.

5.1 Firewall Configuration Results

A total of **five custom inbound firewall rules** were successfully implemented.

Three rules were configured to allow secure network communication, while two rules were created to block insecure legacy protocols.

This configuration follows the **Principle of Least Privilege**, where only required network services are permitted while unnecessary services are denied.

Rule Name	Port	Action	Status
CG_HTTP_Allow	80	Allow	Successfully Implemented
CG_HTTPS_Allow	443	Allow	Successfully Implemented
CG_SSH_Allow	22	Allow	Successfully Implemented
CG_FTP_Block	21	Block	Successfully Implemented
CG_Telnet_Block	23	Block	Successfully Implemented

Table 5.1: Firewall Rule Implementation Results

5.2 Windows Security Hardening Results

The Windows operating system was evaluated after completing the hardening process. The assessment confirmed that Microsoft Defender Antivirus, Windows Defender Firewall, Secure Boot, TPM, Device Encryption, Core Isolation, Memory Integrity, Windows Update, and User Account Control were all functioning correctly. Furthermore, the Guest account remained disabled, reducing the overall attack surface of the workstation.

Security Feature	Before	After
Firewall	Enabled	Verified
Defender Antivirus	Enabled	Verified
Secure Boot	Unknown	Verified
TPM	Unknown	Verified
Device Encryption	Unknown	Verified
Memory Integrity	Unknown	Verified
Windows Update	Checked	Operational
UAC	Default	Verified
Guest Account	Unknown	Disabled

Table 5.2 – Windows Security Assessment

5.3 Security Improvements Achieved

The project successfully strengthened endpoint security through multiple defensive layers. The implemented security improvements include:

- Reduced exposure to insecure network services.
- Protection against unauthorized inbound connections.
- Enhanced malware protection using Microsoft Defender Antivirus.
- Secure boot process validation.
- Hardware-backed protection using Trusted Platform Module (TPM).
- Kernel-level protection through Memory Integrity.
- Improved protection of stored information using Device Encryption.
- Protection against unauthorized administrative changes using User Account Control.
- Reduced attack surface through Guest account verification.
- Comprehensive verification of all implemented security controls.

Security Area	Improvement
Network Security	Improved
Endpoint Protection	Improved
Operating System Security	Improved
Authentication Security	Improved
Data Protection	Improved
Hardware Security	Improved
Overall Security Posture	Significantly Improved

Table 5.3 – Security Improvements

5.4 Security Score Assessment

Based on the implemented firewall configuration and Windows hardening measures, the assessment indicates that the workstation meets the majority of Microsoft's recommended endpoint security practices. The overall security posture was evaluated using the implemented security controls and verification results.

Category	Status
Firewall Protection	Excellent
Endpoint Protection	Excellent
System Hardening	Excellent
Device Security	Excellent
Encryption	Excellent
Authentication	Excellent
Overall Rating	9.5 / 10

Table 5.4 – Overall Security Assessment

5.5 Discussion

The implementation demonstrated that Windows 11 provides a comprehensive set of built-in security controls capable of protecting enterprise workstations without requiring additional third-party software. The combination of firewall configuration, endpoint protection, hardware-backed security, and operating system hardening significantly reduced the system's attack surface.

The verification process confirmed that all firewall rules were correctly configured and that Windows Defender Firewall continued protecting all network profiles. Likewise, Microsoft Defender Antivirus, Secure Boot, TPM, Core Isolation, Device Encryption, Windows Update, and User Account Control operated as expected throughout the assessment.

Although this project focused on a single Windows workstation, the same methodology can be applied across enterprise environments to establish secure baseline configurations for multiple devices.

Chapter Summary

This chapter analyzed the outcomes of the implemented firewall configuration and Windows system hardening activities. The assessment confirmed that all security controls were successfully implemented and verified, resulting in a significantly improved security posture. The findings demonstrate the effectiveness of Windows native security technologies in protecting enterprise endpoints.

CHAPTER 6

6. SECURITY BEST PRACTICES & RECOMMENDATIONS

Firewall configuration and operating system hardening are continuous security processes rather than one-time activities. Organizations should regularly review their security configurations, monitor firewall rules, install security updates, and verify compliance with industry security standards. Proper maintenance of endpoint security significantly reduces the likelihood of cyberattacks and strengthens an organization's overall security posture.

The following recommendations are based on Microsoft Security Baselines, the Center for Internet Security (CIS) Benchmarks, and the National Institute of Standards and Technology (NIST) Cybersecurity Framework.

6.1 Firewall Management Best Practices

A well-configured firewall serves as the first line of defense against unauthorized network access. Firewall rules should be reviewed periodically to ensure that only required services remain accessible. Obsolete or unused firewall rules should be removed promptly to minimize the attack surface.

Organizations should also document every firewall rule, clearly defining its purpose, associated service, and justification. Monitoring firewall logs regularly helps identify suspicious traffic and supports timely incident response.

Best Practice	Description
Allow only required ports	Reduce unnecessary exposure
Block unused services	Minimize attack surface
Review firewall rules regularly	Remove obsolete rules
Enable firewall logging	Improve monitoring and auditing
Apply rules to all profiles	Maintain consistent protection
Document firewall changes	Improve change management

Table 6.1: Firewall Security Best Practices

6.2 Windows System Hardening Best Practices

System hardening involves configuring Windows security features to reduce vulnerabilities and improve system resilience. Organizations should enable built-in Windows security technologies and ensure that all security features remain operational through periodic verification.

Regular operating system updates, strong authentication policies, secure boot mechanisms, encryption, and hardware-backed security technologies collectively strengthen endpoint protection.

Recommendation	Security Benefit
Enable Microsoft Defender Antivirus	Malware protection
Enable Secure Boot	Prevent boot-level attacks
Enable TPM	Hardware-backed trust
Enable Device Encryption	Protect stored data
Enable Memory Integrity	Prevent kernel attacks
Keep Windows Updated	Patch vulnerabilities
Disable unused accounts	Reduce attack surface
Keep UAC Enabled	Prevent unauthorized changes

Table 6.2 – Windows Hardening Recommendations

6.3 Enterprise Security Recommendations

Based on the implementation and assessment performed during this project, the following recommendations are suggested for organizations deploying Windows workstations:

- Implement the Principle of Least Privilege for user accounts.
- Use Multi-Factor Authentication (MFA) wherever possible.
- Maintain regular operating system and software updates.
- Monitor Windows Defender Firewall logs for suspicious activity.
- Conduct periodic firewall rule reviews and remove obsolete rules.
- Deploy endpoint detection and response (EDR) solutions in enterprise environments.
- Encrypt all corporate devices using BitLocker or Device Encryption.
- Implement centralized security monitoring using SIEM platforms.

Windows Firewall Configuration & System Hardening

- Perform regular security awareness training for users.
- Conduct periodic vulnerability assessments and security audits.

Security Area	Recommendation
Authentication	Multi-Factor Authentication (MFA)
Endpoint Protection	Microsoft Defender / EDR
Network Security	Firewall Monitoring
Patch Management	Automatic Updates
Data Protection	Full Disk Encryption
Security Monitoring	SIEM Deployment
User Awareness	Cybersecurity Training

Table 6.3 – Enterprise Security Recommendations

6.4 Mapping to Industry Standards

The security controls implemented during this project align with widely accepted cybersecurity frameworks and best practices.

Security Control	Microsoft Security Baseline	CIS Controls	NIST CSF
Windows Firewall	✓	✓	Protect (PR)
Microsoft Defender Antivirus	✓	✓	Detect (DE)
Secure Boot	✓	✓	Protect (PR)
TPM	✓	✓	Protect (PR)
Device Encryption	✓	✓	Protect (PR)
Windows Update	✓	✓	Identify / Protect
User Account Control	✓	✓	Protect (PR)

Table 6.4 – Security Framework Mapping

6.5 Future Enhancements

Although the implemented security controls significantly strengthened the Windows workstation, additional security measures could further improve the overall security posture in enterprise environments.

Potential future enhancements include:

- Integration with Microsoft Defender for Endpoint.
- Deployment of a Security Information and Event Management (SIEM) solution.
- Centralized firewall policy management using Group Policy.
- Automated compliance auditing using PowerShell scripts.
- Endpoint Detection and Response (EDR) deployment.
- Vulnerability scanning using Microsoft Defender Vulnerability Management or Nessus.
- Continuous security monitoring and alerting.

Chapter Summary

This chapter presented security best practices and recommendations for maintaining a secure Windows environment. The proposed recommendations are aligned with Microsoft Security Baselines, CIS Controls, and the NIST Cybersecurity Framework. Implementing these practices can further strengthen endpoint security and improve organizational resilience against evolving cyber threats.

CHAPTER 7

7. CHALLENGES, LEARNING OUTCOMES AND CONCLUSION

The successful completion of this project required practical implementation, troubleshooting, verification, and documentation of multiple Windows security features. Throughout the project, several challenges were encountered while configuring firewall rules, verifying security settings, and collecting technical evidence. Overcoming these challenges provided valuable practical experience in Windows security administration and endpoint hardening.

This chapter summarizes the challenges faced, the knowledge gained, and the overall outcomes of the project.

7.1 Challenges Faced

During the implementation of this project, several technical and operational challenges were encountered. These challenges were resolved through systematic troubleshooting, Microsoft's official documentation, Windows administrative tools, and practical experimentation.

The major challenges included:

- Understanding the differences between inbound and outbound firewall rules.
- Identifying the correct Windows Defender Firewall interface for rule creation.
- Verifying firewall rules using both Command Prompt and Windows PowerShell.
- Understanding the relationship between firewall rules and port filters.
- Verifying advanced Windows security features such as Secure Boot, TPM, Memory Integrity, and Device Encryption.
- Organizing screenshots and maintaining proper technical documentation.
- Interpreting PowerShell outputs and validating security configurations.

Successfully resolving these challenges improved practical troubleshooting skills and increased confidence in Windows security administration.

Challenge	Solution
Firewall rule creation	Used Windows Defender Firewall with Advanced Security
PowerShell command execution	Executed commands with Administrator privileges

Challenge	Solution
Verification of firewall rules	Used netsh and PowerShell cmdlets
Security feature verification	Verified through Windows Security and PowerShell
Documentation	Organized screenshots and added captions

Table 7.1: Challenges Encountered and Solutions Implemented

7.2 Learning Outcomes

This project significantly improved practical knowledge of Windows endpoint security and firewall management. Rather than focusing only on theoretical concepts, the project emphasized hands-on implementation, verification, and documentation of security controls.

The key learning outcomes include:

- Understanding Windows Defender Firewall architecture.
- Creating and managing custom firewall rules.
- Allowing and blocking network services based on organizational requirements.
- Using Windows PowerShell for firewall verification.
- Performing Windows operating system hardening.
- Verifying Microsoft Defender Antivirus configuration.
- Understanding Secure Boot, TPM, Device Encryption, and Memory Integrity.
- Applying security best practices for Windows endpoints.
- Producing professional technical documentation.

Skill	Level Achieved
Windows Firewall Administration	Advanced Beginner
Windows Security	Intermediate
PowerShell	Intermediate
System Hardening	Intermediate
Security Documentation	Intermediate
Endpoint Security	Intermediate

Table 7.2 – Skills Developed

7.3 Project Outcomes

The project successfully achieved all predefined objectives established during the planning phase.

The Windows Defender Firewall was configured with custom inbound rules, insecure services were blocked, firewall functionality was verified using multiple administrative tools, and several Windows security features were validated.

The implementation demonstrated that Windows native security technologies provide comprehensive protection for endpoint devices without requiring additional third-party security software.

Objective	Status
Baseline Assessment	Completed
Firewall Configuration	Completed
Custom Rule Creation	Completed
Firewall Verification	Completed
Windows Hardening	Completed
Security Validation	Completed
Documentation	Completed

Table 7.3 – Project Objectives Achievement

7.4 Conclusion

This project successfully demonstrated the practical implementation of Windows Defender Firewall configuration and Windows operating system hardening using native Microsoft security technologies. A structured methodology was followed to assess the baseline environment, configure custom firewall rules, verify firewall functionality, and implement essential security hardening measures.

The firewall configuration effectively allowed secure services while blocking insecure protocols, following the Principle of Least Privilege. Multiple Windows security mechanisms including Microsoft Defender Antivirus, Secure Boot, Trusted Platform Module (TPM), Core Isolation, Memory Integrity, Device Encryption, Windows Update, User Account Control, and Guest Account management were verified to strengthen endpoint protection.

The verification process using Windows Security, Command Prompt, and PowerShell confirmed that all implemented security controls were functioning correctly. The project demonstrated how layered security controls can significantly improve the security posture of a Windows workstation and reduce the overall attack surface.

Overall, this project enhanced practical cybersecurity skills related to firewall administration, Windows endpoint protection, system hardening, PowerShell verification, and technical documentation. The knowledge and experience gained through this implementation provide a strong foundation for future work in cybersecurity, system administration, and security operations.

Project Component	Status
Firewall Configuration	Successfully Completed
Firewall Verification	Successfully Completed
Windows Hardening	Successfully Completed
Security Assessment	Successfully Completed
Documentation	Successfully Completed
Overall Project	Successfully Completed

Table 7.4 – Overall Project Summary

Professional Closing Statement

"Effective cybersecurity begins with secure configurations. A properly hardened system is the first line of defense against evolving cyber threats."

Chapter Summary

This chapter summarized the challenges encountered, learning outcomes, project achievements, and overall conclusions. The project successfully met all objectives and demonstrated the practical implementation of Windows Defender Firewall configuration and Windows system hardening using native Microsoft security technologies.

REFERENCES

The following references were consulted during the implementation and documentation of this project.

1. Microsoft. (2025). *Microsoft Defender Firewall documentation*.
<https://learn.microsoft.com/windows/security/operating-system-security/network-security/windows-firewall/>
2. Microsoft. (2025). *Microsoft Defender Antivirus documentation*.
<https://learn.microsoft.com/defender-endpoint/>
3. Microsoft. (2025). *Windows Security documentation*.
<https://support.microsoft.com/windows>
4. Microsoft. (2025). *PowerShell NetSecurity Module*.
<https://learn.microsoft.com/powershell/module/netsecurity/>
5. Microsoft. (2025). *Device Encryption in Windows*.
<https://support.microsoft.com/windows/device-encryption>
6. Microsoft. (2025). *Secure Boot*. <https://learn.microsoft.com/windows/security>
7. National Institute of Standards and Technology (NIST). (2024). *Cybersecurity Framework (CSF) 2.0*. <https://www.nist.gov/cyberframework>
8. Center for Internet Security (CIS). (2024). *CIS Microsoft Windows Benchmarks*.
<https://www.cisecurity.org/cis-benchmarks>
9. OWASP Foundation. (2024). *OWASP Top 10 Web Application Security Risks*.
<https://owasp.org/www-project-top-ten/>
10. Microsoft. (2025). *Windows PowerShell Documentation*.
<https://learn.microsoft.com/powershell/>
11. Microsoft. (2025). *Windows Security Baselines*.
<https://learn.microsoft.com/windows/security>
12. Microsoft. (2025). *Windows Update Documentation*.
<https://support.microsoft.com/windows/windows-update>

Reference Note

The implementation procedures described in this report were performed on a Windows 11 system using native Microsoft security features. Industry standards and official documentation from Microsoft, NIST, CIS, and OWASP were consulted to ensure alignment with recommended cybersecurity practices.

APPENDIX

This appendix contains the primary commands, abbreviations, and evidence used during the implementation and verification of Windows Defender Firewall configuration and Windows system hardening.

Appendix A – Commands Used

Command	Purpose
hostname	Display system hostname
whoami	Display current user
ipconfig /all	Display network configuration
netsh advfirewall firewall show rule name=all	Verify firewall rules
Get-NetFirewallRule	Verify custom firewall rules
Get-NetFirewallPortFilter	Verify configured ports
Get-NetFirewallProfile	Verify firewall profiles
Get-MpComputerStatus	Verify Microsoft Defender status
Confirm-SecureBootUEFI	Verify Secure Boot
net user	Display local user accounts
net user Guest	Verify Guest account status

Table A.1 – Commands Executed During the Project

Appendix B – Abbreviations

Abbreviation Meaning

UAC	User Account Control
TPM	Trusted Platform Module
HVCI	Hypervisor-Protected Code Integrity
LSA	Local Security Authority
GUI	Graphical User Interface
CLI	Command Line Interface
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
SSH	Secure Shell
FTP	File Transfer Protocol
TCP	Transmission Control Protocol
SIEM	Security Information and Event Management
CIS	Center for Internet Security
NIST	National Institute of Standards and Technology

Table A.2 – List of Abbreviations

